Dataset & Constraint Interaction Complexity

Summary

Our modeling dataset lacks a few important large noun classes and therefore exhibits a simplified constraint interation. It therefore cannot reflect the actual complexity of German nominal compounding, and cannot be used to build the final German compounding model. However, it will still provide a favorable ground for the first attempts of modeling German compounding. First, the simplified constraint interaction will narrow down the subject of study: by focusing on a single morphology vs. semantics interaction, we will not be overwhelmed by the diversity and the number of constraint conflicts. Second, all N1s in the dataset may attach both a pluralic and a non-pluralic linker, thanks to Schäfer & Pankratz 2018. This alternation will make a more representative test of the model because it requires an accurately calibrated apparatus to balance between the two cases for the same input. Third, the small size of the dataset will allow for more detailed inspection of the results. In sum, despite the cons of the dataset, we will be able to produce first insights about constraint conflict resolution with it, and work out a direction for future research. We therefore believe that our modeling dataset is a great baseline for the initial findings.

Our modeling dataset consists of ~900 compounds (36 unique N1s) with all implemented constraints defined over them. We derived it by intersecting the items of *GeCoDB_v07_GN* and *SP18_Pl_Constr*.

Since Schäfer & Pankratz 2018 targeted nouns that exhibit an alternation between a pluralic and a non-pluralic linker, *SP18_Pl_Constr* (hence, our modeling dataset) mostly contains simplex masculine and neutral nouns, which prevail among compounds that satisfy this condition. This leads to a noticeable limitation of the dataset: phonological and especially derivational generative constraints are largely underused in it. Thus, only 50% and 20% of these constraints, respectively, enter the modeling of this dataset.

```
Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.
```

	gen			restr		
	%	ours	reference	%	ours	reference
morphological	75	15	20	100	8	8
derivational	20	2	10	100	7	7
semantic	81	9	11	100	0	0
phonological	50	4	8	85	6	7
lexical	100	0	0	100	1	1

Given that phonological and derivational constraints are the most productive source of *-s-*, this linker is underrepresented in the dataset.

```
Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.
```

```
_          0.570
_+en_      0.186
_+=er_     0.160
_+e_       0.048
_+=e_      0.015
_+s_       0.015
_+=_       0.007
dtype: float64
```

Note that the disproportion becomes striking when we consider the actual involvement of these constraints (cf. analogy of lemma vs. token). The "token usage" of the morphological generative constraints at least doubles all the other domain types combined. Semantic constraints are the second most "token-frequent" domain type. Phonology contributes to linker generation slightly, and derivation negligibly little.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

	gen	restr
morphological	1816	4180
semantic	498	0
phonological	264	1707
derivational	20	1756
lexical	0	16

Quantitative Overview of Constraint Conflicts

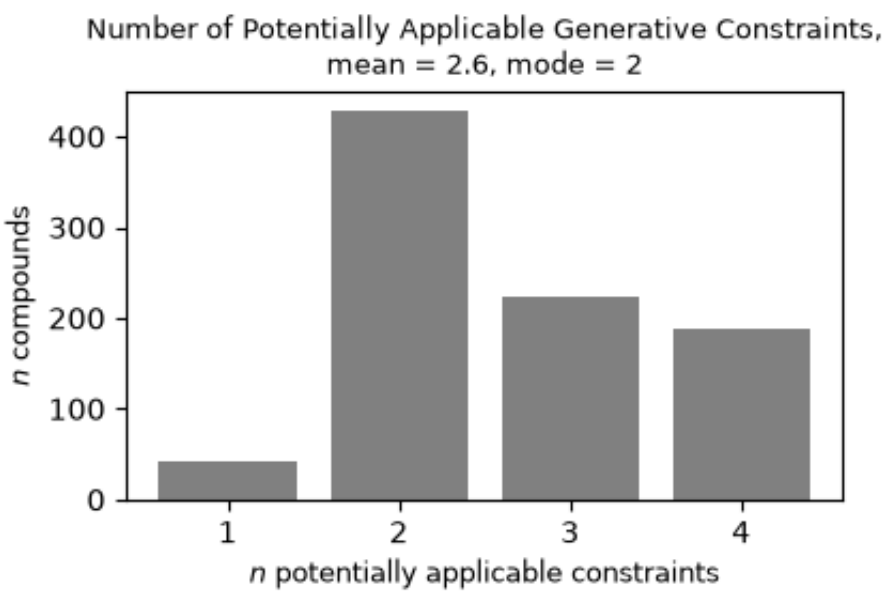
Summary

On average, a compound has $2.6$$2.6$ generative and $5.3$$5.3$ restrictive linkers potentially applicable to it. Their joint potential application leads to $1.7$$1.7$ linkers acceptable in a compound. The results are to interpreted in the context of the simplified constraint interaction.

The absolute majority of the compounds are covered by 2-4 generative constraints, with mean of $2.6$$2.6$ and mode of $2$$2$.

Not all of these constraints necessarily conflict within one compound. On average, every $\sim 1.5$$\sim 1.5$ constraints potentially applicable to one compound actually generate the same linker. This begs the question whether these constraints cooperate during linker generation and jointly increase the probability of their linker. On the other hand, some constraints may return multiple generations. We will henceforth refer to this case as *multi-linker* constraints, as opposed to *targeted* constraints.

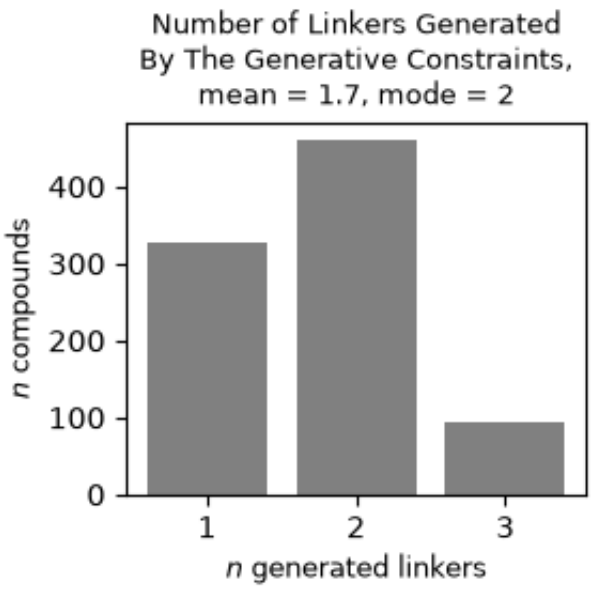
Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.



In a considerable amount of compounds, the (potentially) cooperating constraints converge to a single generated linker. Still, the constraints generate two linkers for most of the compounds, and even three for some of them (mean, $1.7$$1.7$, mode $2$$2$).

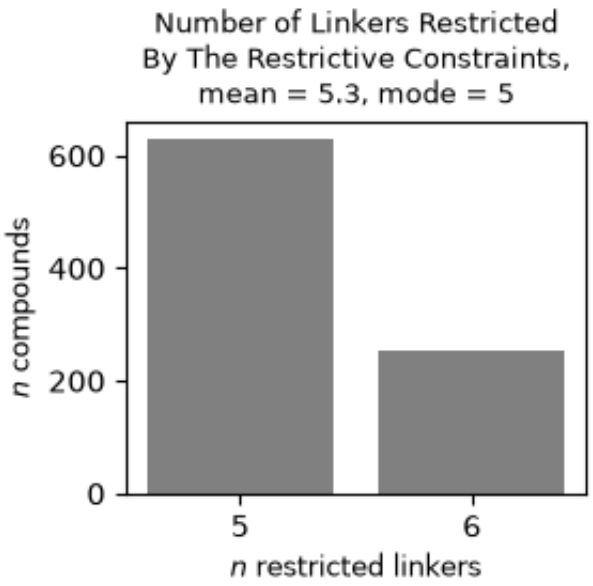
Remember that these numbers are observed over a modeling dataset with a simplified constraint interaction. The real picture must be more intricate. But again, simplicity is a virtue at this initial attempt to model German compounding: it will be much easier to grasp the first patterns of it from the two-linker outputs.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.



At the same time, the number of restricted linkers is substantially larger and is 5 or 6 for all the compounds (mean $5.3$$5.3$, mode $5$$5$).

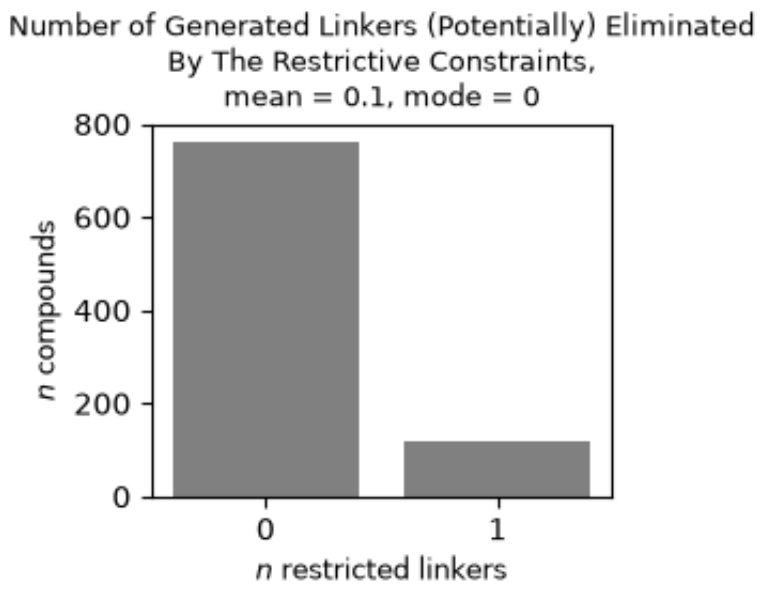
Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.



And still, restrictive linkers may only (potentially!) eliminate a maximum of one linker in a very small number of compounds (mean $0.1$$0.1$, mode $0$$0$).

The explanation to that lies in the fact that most of the restrictive constraints are paradigm-based and eliminate non-paradigmatic linkers. But since most of the linkers can only be generated paradigmatically in the first place, there is virtually almost no need for these conditions. The restrictive constraints do really come largely into handy when it concerns the *-s-*\linker. Our dataset is just unsuitable to test that because, as discussed, the *-s-* is underrepresented in our data.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.



On The Paradox of Semantic Constraints

Summary

Being very irregular, semantic constraints are still very important in German compounding because they are the only ones that generate (most) pluralic linkers.

In our dataset, the morphology vs. semantics conflict is the cornerstone of constraint interaction. It results in the conflict of type *-0-* vs. pluralic linker being the predominant linker conflict in our compounds.

Pluralic linkers *-e-*, *-"e-*, and *-"0-* are only ever produced by targeted semantics-involving constraints. Linker *-"er-* is produced by two targeted semantics-involving constraint and one multi-linker non-semantic constraint. Linker *-en-* is produced by two targeted semantics-involving constraints, one targeted non-semantic constraints, and two multi-linker non-semantic constraints. In sum, four out of five pluralic linkers are only produced by semantic constraints if we ignore multi-linker constraints. If we include multi-linker constraints back into account, three pluralic linkers are still only produced by semantic constraints. In other words, if we exclude semantics-involving constraints from consideration, we will realistically only be able to produce *-0-*, *-s-*, *-en-*, and maybe *-"er-*.* Indeed, in a dedicated substudy, we predicted the linkers for the entries of the *GeCoDB_v07_GN* dataset, for which the semantic constraints are undefined, using the Albright & Hayes 2002 method. From ~49k trials, only *-0-*, *-s-*, and *-en-* were ever produced.

This poses a paradox and a modeling challenge for nominal compounding. While the semantic constraints are much less regular, and semantics seems to be only a soft cue in German compounding, it is responsible for at least half of the linker inventory. While the number of the affected compounds is relatively small, it is non-negligible and, and these compounds cannot be conveniently accomodated as exceptions.

	morphological		morphological+phonological		morphological+phonological+semantic	...	morphological+semantic		derivational		phonological	
	n	constraints	n	constraints	n	...	constraints	n	constraints	n	constraints	
_	5	p2l:decl_cl:pl:#er-0/er|def-0/er, p2l:decl_cl:pl:#e_uml-0|def-0, p2l:decl_cl:pl:#0_uml-0|def-0, p2l:decl_cl:pl:#e-0|def-0, p2l:decl_cl:mixed-0/s/en|def-0/s/en	1	p2l:phon_fin:weak_F_cons$-0/s/en|def-0/s/en	0	...	p2l:decl_cl:pl:#er-0/er|mass-interpr-0, p2l:decl_cl:pl:#er-0/er|!pl_interpr-0	0		1	p2l:phon_fin:sibilant$-0
_+=_	0		0		0	...	p2l:decl_cl:pl:#0_uml-0|pl_interpr-0_uml	0		0	
_+=e_	0		0		0	...	p2l:decl_cl:pl:#e_uml-0|pl_interpr-e_uml	0		0	
_+=er_	1	p2l:decl_cl:pl:#er-0/er|def-0/er	0		0	...	p2l:decl_cl:pl:#er-0/er|pl_interpr-er, p2l:decl_cl:pl:#er-0/er|anim-er	0		0	
_+e_	0		0		0	...	p2l:decl_cl:pl:#e-0|pl_interpr-e	0		0	
_+en_	1	p2l:decl_cl:mixed-0/s/en|def-0/s/en	1	p2l:phon_fin:weak_F_cons$-0/s/en|def-0/s/en	1	...	p2l:decl_cl:mixed-0/s/en|pl_interpr-en	0		1	p2l:phon_fin:schwa$-en|def-en
_+s_	1	p2l:decl_cl:mixed-0/s/en|def-0/s/en	1	p2l:phon_fin:weak_F_cons$-0/s/en|def-0/s/en	0	...		2	p2l:drv:sfx:sfx-s|def-s, p2l:drv:prx_deverb-s	0		

7 rows × 12 columns

After removing semantic constraints
there are no constraints that produce _+=_, _+=e_, and _+e_.

Python

	morphological		morphological+phonological		derivational		phonological	
	n	constraints	n	constraints	n	constraints	n	constraints
_	5	p2l:decl_cl:pl:#er-0/er|def-0/er, p2l:decl_cl:pl:#e_uml-0|def-0, p2l:decl_cl:pl:#0_uml-0|def-0, p2l:decl_cl:pl:#e-0|def-0, p2l:decl_cl:mixed-0/s/en|def-0/s/en	1	p2l:phon_fin:weak_F_cons$-0/s/en|def-0/s/en	0		1	p2l:phon_fin:sibilant$-0
_+=_	0		0		0		0	
_+=e_	0		0		0		0	
_+=er_	1	p2l:decl_cl:pl:#er-0/er|def-0/er	0		0		0	
_+e_	0		0		0		0	
_+en_	1	p2l:decl_cl:mixed-0/s/en|def-0/s/en	1	p2l:phon_fin:weak_F_cons$-0/s/en|def-0/s/en	0		1	p2l:phon_fin:schwa$-en|def-en
_+s_	1	p2l:decl_cl:mixed-0/s/en|def-0/s/en	1	p2l:phon_fin:weak_F_cons$-0/s/en|def-0/s/en	2	p2l:drv:sfx:sfx-s|def-s, p2l:drv:prx_deverb-s	0	

*Note that this would have been the case even if all the constraints were active over the dataset. The missing constraints can only produce *-0-*, *-s-*, and *-en-*, themselves.

To demonstrate the generative role of the semantic constraints, we scrutinize the constraint conflicts for each N1 in our dataset.

Out of 36 N1s in our dataset, ten exhibit an alternation of the number of potentially applicable constraints in different compounds.

- N1s 'Mutter', 'Vogel', 'Stadt', 'Weg', 'Zahn' are only covered by the default constraint and a morphological constraint that both predict *-0-* in the basic case. N1 'Geschenk' belongs to the same case with an additional derivational constraint that generates *-s-*. However, in compounds in which they have a plural reading, a semantic constraint enters the conflict and additionally generates a pluralic linker. For these six N1s, the pluralic linker is *-e-*, *-"e-*, or *-"0-*. As discussed above, involving semantic constraints is the only way to generate these three pluralic linkers. These six N1s are a very representative case thereof. Indeed, for these N1s, only *-0-* is generated without a semantic constraint, and both *-0-* and a pluralic linker are generated otherwise.
- N1s 'Bett', 'Person', and 'Schwester' are an example of a semantic constraint being the only targeted constraint for *-en-* (except for the schwa-constraint which is, obviously, not applicable here).
- The *-"er-*\plural N1s always come with a morphological and a semantic constraint. As opposed to N1s of the other plural types (which only have a constraint for a pluralic reading), the *-"er-*\plural N1s also have one for a singular reading. That is the reason these N1s do not show such an alternation of the number of potentially applicable constraints. Still, the tenth N1 'Ei' is special because it may involve another, non-pluralic semantic constraint, which, however, does not change the generated linkers.

Now keep in mind that further similar N1s may exhibit the same behavior and just don't in our specific dataset. Thus, N1 'Vater' only builds compounds with a singular N1 reading in our dataset, which is why it never interacts with a pluralic constraint. However, there are cases like 'Väterverein' identical to 'Mütterzentrum' where *-"0-* is generated by the pluralic constraint. Had the compound 'Väterverein' been in the dataset, we would have had the same alternation for N1 'Vater'. This is important insofar that *-e-*, *-"e-*, or *-"0-* can only be generated by the pluralic semantic constraints, while the restriction of those can only succeed on the unparadigmaticity grounds. Consequently, *-e-*, *-"e-*, and *-"0-*\plural N1s with a singular reading can neither generate, nor restrict these linkers. This is exactly what we see in "semantics-free" cases of 'Vogel', 'Stadt', 'Vater' etc.: their pluralic linkers are undefined. They may have it because it is not restricted (it is paradigmatic) but it is never generated because no necessary pluralic semantic constraint is involved. An unrelated case of undefined linkers is provided by N1s 'Frau' and 'Gerät'. While these cases are of a low relevance at this iteration, it might be reasonable to analyze them in detail in future.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

		generated_linkers	restricted_linkers	undefined_linkers	example	
bett	**2**	-0-, -en-, -s-	-s-, -e-, -"er-, -"e-, -"0-		bett_wäsche	def-0, p2l:decl_
	3	-0-, -en-, -s-	-s-, -e-, -"er-, -"e-, -"0-		bett_+en_zahl	def-0, p2l:decl_ p2l:decl_cl
ei	**3**	-0-, -"er-	-s-, -en-, -e-, -"e-, -"0-		ei_gelb	def-0, p2l:decl_ p2l:
	4	-0-, -"er-	-s-, -en-, -e-, -"e-, -"0-		ei_sprung	def-0, p2l:decl_ p2l:(p2l:decl_c
geschenk	**3**	-0-, -s-	-en-, -e-, -"er-, -"e-, -es-, -"0-		geschenk_idee	def-(
	4	-0-, -e-, -s-	-en-, -e-, -"er-, -"e-, -es-, -"0-		geschenk_+e_liste	def-(p2l:(
mutter	**2**	-0-	-s-, -en-, -e-, -"er-, -"e-, -es-	_+=_	mutter_sprache	def-0, p2
	3	-0-, -"0-	-s-, -en-, -e-, -"er-, -"e-, -es-		mutter_+=_treff	def-0, p2l p2l:decl_cl:pl:
person	**2**	-0-, -en-, -s-	-s-, -e-, -"er-, -"e-, -es-, -"0-		person_+en_gesellschaft	p2l:phon_fin:weak_
	3	-0-, -en-, -s-	-s-, -e-, -"er-, -"e-, -es-, -"0-		person_+en_kreis	p2l:phon_fin: p2l:phon_fin:weak_
schwester	**2**	-0-, -en-, -s-	-s-, -e-, -"er-, -"e-, -es-, -"0-		schwester_modell	p2l:phon_fin:weak_

Finally, the unexpected importance of the semantic constraint is clearly visible on the domain conflict heatmap below. In compounds for which the constraints generate exactly two linkers, the heatmap visualizes the conflicts of domains involved to generate these two linkers. The default constraint is ignored. The $x$$x$ and the $y$$y$ axes mark the domains that generate the first and the second linker, respectively. The domains that generate one linker are gathered as a union of the domains of all the constraints that generate this linker. For example, if in a compound constraint $C1$$C1$ (morphological) and a constraint $C2$$C2$ (phonological) generate linker $L1$$L1$, and a constraint $C3$$C3$ (derivational) generates linker $L2$$L2$, then the record for this compound on the heatmap will be "morphological+phonological" vs. "derivational". We only concentrate on two-linker generations because the one-linker generations are of no interest, and all of the three-linker generations in our dataset originate from multi-linker constraints (the mixed nouns constraint: 'Bett', the final-consonant feminine noun constraint: 'Person', 'Nation' etc.)*.

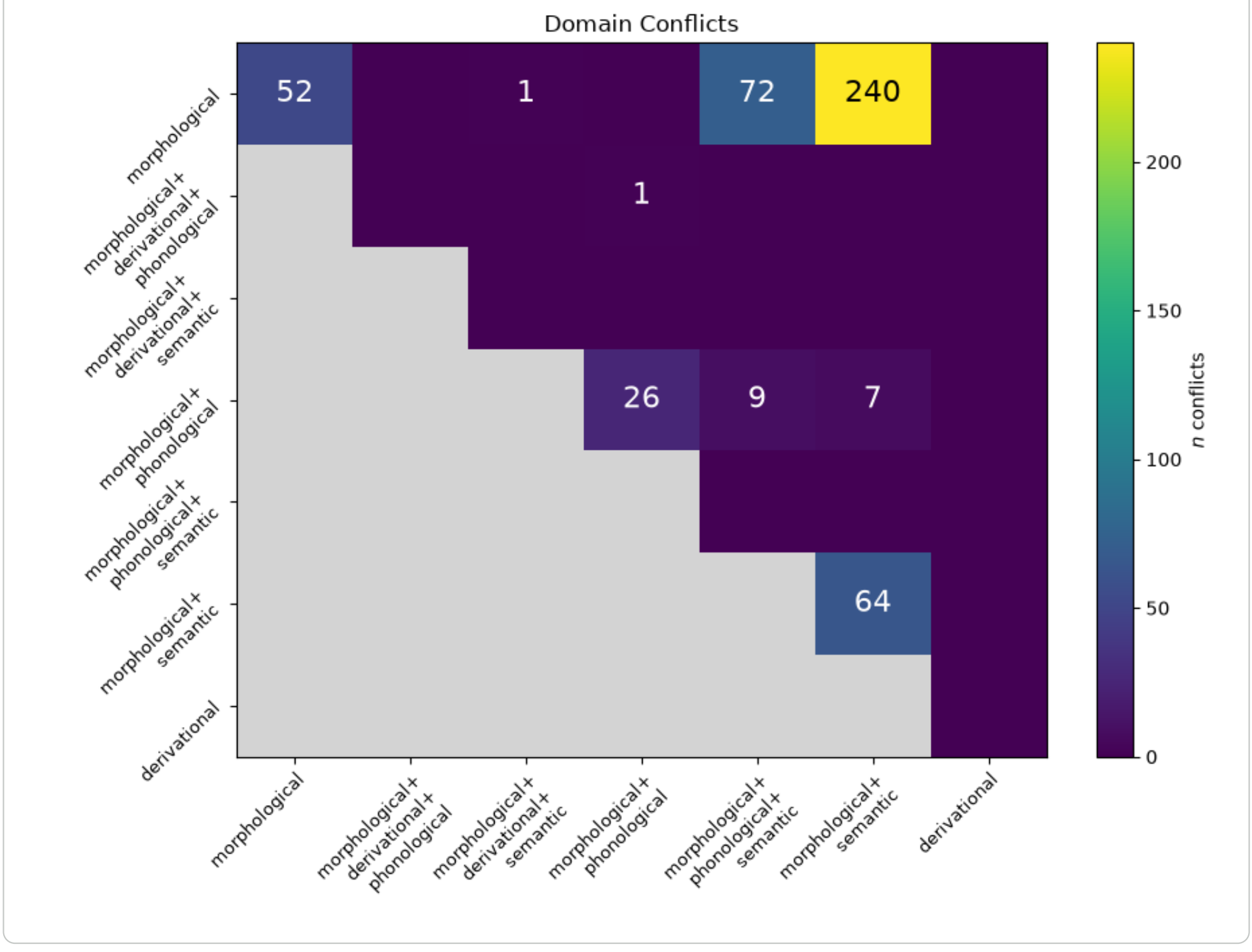
As pointed out above, morphology and semantics play the central role in modeling our dataset. This is confirmed by the fact that the absolute majority of the two-linker generations are born in the competition of morphology and an alliance of morphology and semantics** -- exactly the 'Mutter', 'Stadt' etc. cases addressed above. A considerable amount is at the intersection of morphology and an alliance of morphology, semantics, and phonology. These are the cases of sibilant-final -*"er*-\plural N1s with a singular interpretation e.g. 'Schloss', 'Haus' where -*"er*- is predicted by the multi-linker morphological -*"er*-\plural constraint. When these nouns have a plural reading, it results in morphology + phonology vs. morphology + semantics conflicts (now the semantic constraint also predicts -*"er*- and not -*0*-). A fascinating case is the morphology + semantics vs. morphology + semantics conflict. It is constituted by -*"er*-\plural N1s with a singular reading that designate animals and persons: 'Kind' and 'Wurm'. While the pluralic semantic constraint strives to assign them a zero linker under the singular reading, another, non-pluralic, semantic constraint predicts -*"er*- because of their animacy.

In sum, we have an overwhelming presence of morphology and semantics in our dataset. These two domains constitute the most of the domain conflicts in it. The semantic constraints are necessary to generate roughly half the linker inventory. At the same time, the paradox of the semantic constraints makes them really irregular at doing so. Thus, the challenge of the compound modeling over our dataset is to somehow reconcile the necessity of the semantic constraints with their low regularity (and coverage). We now turn to the results of our compound modeling to see how good (or bad) our three models are at doing so.

*While the three-linker generations are excluded from the heatmap, we do not consider them irrelevant. The semantic constraints do not predict a dedicated linker within them, indeed, but can surely increase the probability of it. These cases will also be important during compounding modeling. However, the two-linker predictions both make the majority of the cases and are easier to disentangle. They will therefore serve as a more transparent source of interpreting the modeling outcome.

**Which is, however, only logical, because Schäfer & Pankratz 2018 specifically aimed at creating a dataset for investigating this conflict, even though they did not exactly phrase it as a conflict.

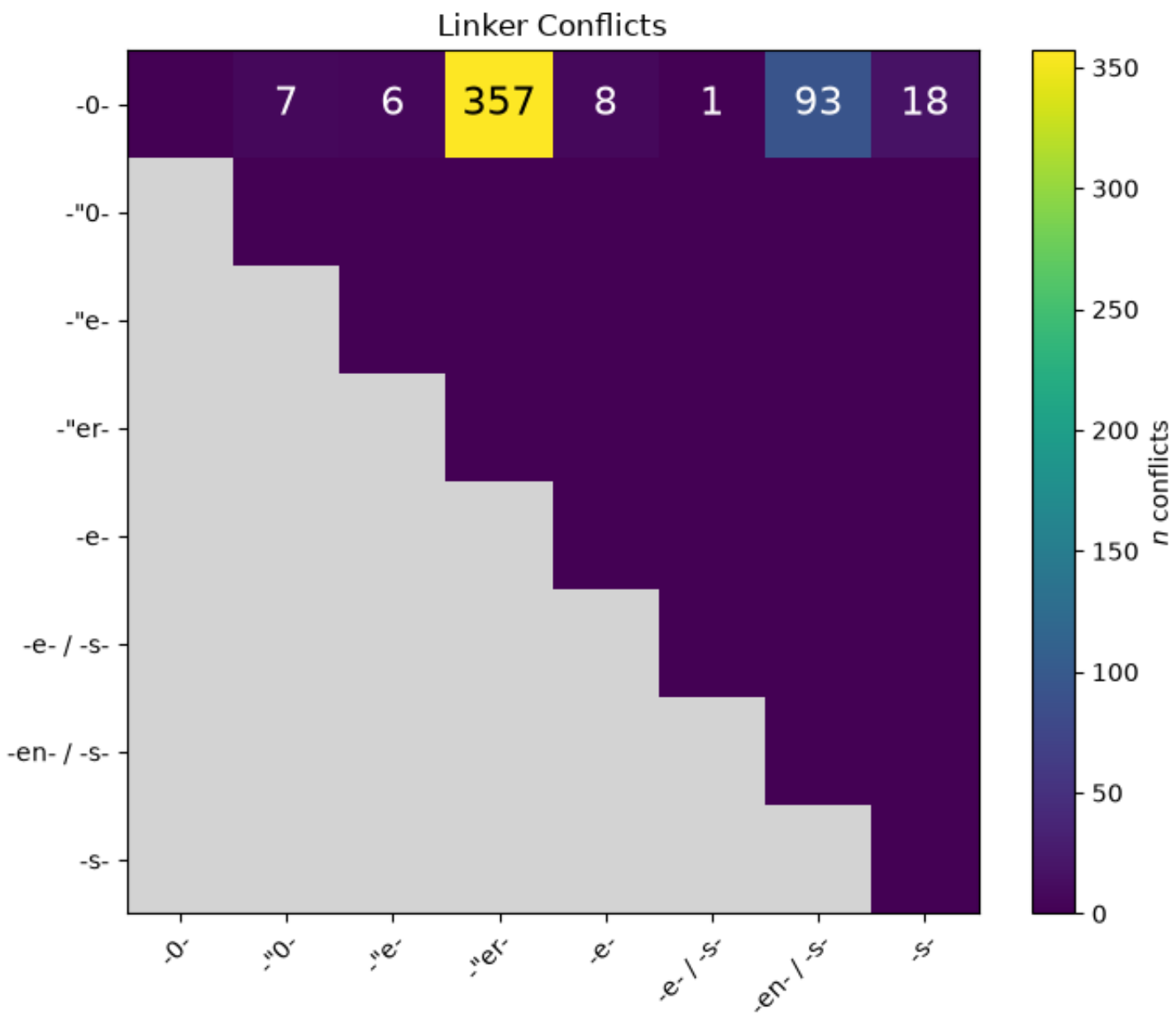
Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.



The overwhelming presence of the morphology vs. semantics conflict empirically leads to the dominance of *-0-* vs. pluralic linker conflicts. In all cases except for $18$$18$, *-0-* is generated alongside a pluralic linker.

Note the inflated number of the conflicts of *-0-* with *-"er-* and *-en-* (/ *-s-*). It is caused by the fact that these explicit linkers are generated regardless of the semantics by multi-linker morphological constraints. Such compounds are also the most constraint-dense in our dataset (see N1s vs. constraints above). While not diminishing the importance of the other linkers, we should be mindful of the more complicated constraint interaction inside the *-"er-* and *-en-*\plural N1s.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.



Compounding Modeling: Intro

Summary

We have three methods of constraint ordering: by regularity, by coverage, and by condition specificity.

Besides, we have three parameters configurable for each model: the presence of the default constraints in the conflicts, constraint cooperation, and linker elimination.

Following Albright & Hayes 2002 and Yang 2016, we adopt the method of iterative rule (interchangeable with generalizations and constraints in this context) application in modeling the linguistic phenomena. They both argue that a linguistic phenomenon may be governed by a set of rules that are defined over different (possibly overlapping) subsets of the relevant vocabulary, rather then a single rule with a bunch of exceptions. The applicability of these rules is defined by the linguistic properties of the input (see potential applicability in Shmalts 2026). They then demonstrate how these rules compete to produce the output. In a nutshell, both Albright & Hayes 2002 and Yang 2016 first sort the available linguistic rules that govern the target phenomenon by some kind of confidence measure. The rules are then iterated over the input in order of decreasing confidence. Each applicable rule generates a candidate, and at the end the candidate generated by the most confident rule wins. The only methodical difference is, Yang 2016 stops generation at the first applicable rule whereas Albright & Hayes 2002 record all the candidates. For the linker ranking experiments we are planning to conduct in the future, the latter method is clearly much more suitable, as it produces a ranking of the candidates.

Where Albright & Hayes 2002 and Yang 2016 diverge significantly in their methods is the confidence score. The confidence score is designed to resolve rule conflicts by preferring the rule with the higher confidence.

Albright & Hayes 2002 suggest that the regularity (reliability in their own terms; TODO: fact check) of the rule plays the definitive role in conflict resolution. They declare what we call type regularity the raw confidence and derive a final confidence from it by adding the absolute coverage into equation. At this iteration, we do not replicate this step and use item regularity instead. TODO: add justification.

In accordance with the Elsewhere Condition (TODO), Yang 2016 proposes that the order of the rule application follows the decrease in rule specificity, starting with the most specific and finishing with the most general (default) rule*. Yang 2016 himself does not providea definition for specificity. We therefore define two variants of "specificity" ourselves:

1. Coverage specificity. Specificity is simply defined by the number of the covered compounds: the more covered compounds, the less specific the constraint.
2. Condition specificity. Specificity of the constraint is defined by the specificity of the linguistic conditions on its potential applicability. This should take into account both the composition of the conditions, and the specificity of each singular condition in its composition. For example, `[PL:e]` is less specific than `[PL:e] & [sem:PL]`, whereas `[coda:V0]` is less specific than `[coda:e]`.

*While further effects may impact the order of rule application (stem frequency, rule frequency, etc. see Yang 2016), we initially concentrate on specificity alone as the baseline.

We developed the `CompXConstraints` class to implement the iterative rule application and resolve the rule conflicts.

The `method` parameter controls for the employed confidence score: regularity (Albright & Hayes 2002), coverage or condition specificity (Yang 2016).

It is now quite clear if the default constraint actually takes part in the rule competition, or whether it only serves as a backup in case no other rule wins. With parameter `count_def_constr`, we can set whether the default constraint participates in the conflicts.

As discussed above, around $1.5$$1.5$ constraints generate the same linker within one compound. Parameter `constr_cooperation` determines whether such constraints cooperate and try to jointly win the competition, or whether they are in the conflict with each other, after all.

`restrict_with_l2p` is the most important parameter of it all. It tells the model whether to eliminate the linkers generated by the generative constraints, in case they are restricted from generation by the restrictive constraints. In case `restrict_with_l2p` is set `True`, an additional round of linker elimination follows the generation. During the elimination, the generative and the restrictive constraints compete for the (non)existence of the affected linker. Similar to in-generation conflicts, during the generation-restriction conflicts the linker is eliminated if the confidence of the restrictive constraint is higher than that of the generative constraint. Consequently, if the most confident prediction is eliminated, the second most confident linker is returned as the output. This procedure is a novel advancement relative to the referenced Albright & Hayes 2002 and Yang 2016 methods.

Below, we briefly demonstrate the functionality of the `CompXConstraints` class.

```
    cxc = CompXConstraints(appl_ind.loc["wurm_befall"])
```

Before the very prediction, generated (acceptable) and restricted (unacceptable) linkers are already known from the constraints.

```
print("Acceptable linkers:", cxc.acceptable_linkers)
print("Unacceptable linkers:", cxc.unacceptable_linkers)
```

```
Acceptable linkers: defaultdict(<class 'list'>, {'_': ['def-0', 'p2l:decl_cl:pl:#er-0/er|def-0/er', 'p2l
Unacceptable linkers: defaultdict(<class 'list'>, {'_+s_': ['l2p:s|tend_cmpx_morph', 'l2p:s|tend_cmpx_ph
```

The prediction procedure itself only resolves their conflict following the specified parameters. During the prediction, the applicable generative constraints are first sorted by confidence as defined by the prediction method. Here `p2l:decl_cl:pl:#er-0/er|!pl_interpr-0` wins because it has the highers regularity*, hence, confidence.

*While `p2l:decl_cl:pl:#er-0/er|def-0/er` is a more regular constraint, it is a multi-linker constraint. In such cases, we distribute the confidence among the linkers.

```
cxc.predict(
    method="albright-hayes",
    count_def_constr=True,
    constr_cooperation=False,
    eliminate_with_l2p=False
)
cxc.gen_constr_df
```

c_id	linker	condition	spec_score	cvg_score	reg	confidence
p2l:decl_cl:pl:#er-0/erl!pl_interpr-0	_	[PL:er] & [sem:!PL]	25.2	0.829	0.622	0.622
def-0	_	NaN	0.0	0.000	0.573	0.573
p2l:decl_cl:pl:#er-0/erlanim-er	_+=er_	[PL:er] & [[sem:anim] | [sem:pers]]	28.0	0.997	0.501	0.501
p2l:decl_cl:pl:#er-0/erldef-0/er	_	[PL:er]	14.0	0.979	0.815	0.407
p2l:decl_cl:pl:#er-0/erldef-0/er	_+=er_	[PL:er]	14.0	0.979	0.815	0.407

If we do count the default constraint out, it does not participate in the competition.

```
cxc.update_pred_params(count_def_constr=False)
cxc.gen_constr_df
```

c_id	linker	condition	spec_score	cvg_score	reg	confidence
p2l:decl_cl:pl:#er-0/erl!pl_interpr-0	_	[PL:er] & [sem:!PL]	25.2	0.829	0.622	0.622
p2l:decl_cl:pl:#er-0/erlanim-er	_+=er_	[PL:er] & [[sem:anim] | [sem:pers]]	28.0	0.997	0.501	0.501
p2l:decl_cl:pl:#er-0/erldef-0/er	_	[PL:er]	14.0	0.979	0.815	0.407
p2l:decl_cl:pl:#er-0/erldef-0/er	_+=er_	[PL:er]	14.0	0.979	0.815	0.407

The sorted generative constraints are then aggregated in the final linker scoreboard. For that, the most confident constraint is taken for each of the acceptable linkers.

```
cxc.linker_scoreboard
```

linker	confidence	cvg_score	c_id
_	0.622	0.829	p2l:decl_cl:pl:#er-0/erl!pl_interpr-0
_+=er_	0.501	0.997	p2l:decl_cl:pl:#er-0/erlanim-er

The most confident linker is returned as the prediction.

```
cxc.y_pred
```

```
'_'
```

Enabling constraint cooperation sums up the confidences of all the constraints for each linker during the linker scoreboard aggregation. Consequently, if there are a few less regular constraints for one linker, they may cooperatively overtake a more regular singular constraint for another linker. In our case, cooperation enabled *-O-* to win the previously predicted *-en-* (even with the default linker counted out). The *-O-*\linker, by the way, is the correct linker for this compound.

```
cxc.update_pred_params(constr_cooperation=True)
cxc.linker_scoreboard
```

linker	confidence	cvg_score	c_id
_	1.029	1.808	p2l:decl_cl:pl:#er-0/erldef-0/er, p2l:decl_cl:pl:#er-0/erl!pl_interpr-0
_+=er_	0.908	1.976	p2l:decl_cl:pl:#er-0/erldef-0/er, p2l:decl_cl:pl:#er-0/erlanim-er

Finally, if we enable linker elimination with the restrictive constraints, we will have a list of the restricted constraints sorted the same way as the generative constraints. Currently, the constraints are sorted by the regularity.

```
cxc = CompXConstraints(appl_ind.loc["geschenk_+e_liste"])
cxc.predict(
    method="albright-hayes",
    count_def_constr=True,
    constr_cooperation=False,
    eliminate_with_l2p=True
)
cxc.restr_constr_df
```

c_id	linker	condition	spec_score	cvg_score	reg	confidence
l2p:0_uml	_+=_	[PL:uo]	14.0	0.999	1.000	1.000
l2p:e_uml	_+=e_	[PL:ue]	14.0	0.997	1.000	1.000
l2p:er|par	_+=er_	[PL:er]	14.0	0.991	1.000	1.000
l2p:en|par	_+en_	[PL:en]	14.0	0.848	0.985	0.985
l2p:es|mono_syl	_+es_	[nsyl:mono] | [[nsyl:mono] & [drv:PRX]]	20.4	0.990	0.972	0.972
l2p:er|smpx	_+=er_	[drv:smpx]	14.0	0.991	0.903	0.903
l2p:e|smpx	_+e_	[drv:smpx]	14.0	0.991	0.806	0.806

We can see that linker *-e-* predicted by the generative constraints is actually prohibited by one of the restrictive constraints. If the restrictive constraint is more confident, the linker is eliminated, which is documented in the linker scoreboard.

```
cxc.linker_scoreboard
```

linker	confidence	cvg_score	c_id	removed_by
_	0.754	0.838	p2l:decl_cl:pl:#e-0|def-0	
_+s_	0.649	0.841	p2l:drv:prx_deverb-s	
_+e_	0.544	0.921	p2l:decl_cl:pl:#e-0|pl_interpr-e	l2p:e|smpx

Right now, the elimination did not affect the output. However, if the most confident linker is eliminated, the next one takes its place, as will be demonstrated with a coverage-based model below.

```
cxc.update_pred_params(constr_cooperation=False)
cxc.linker_scoreboard
```

linker	confidence	cvg_score	c_id	removed_by
_	0.754	0.838	p2l:decl_cl:pl:#e-0ldef-0	
_+s_	0.649	0.841	p2l:drv:prx_deverb-s	
_+e_	0.544	0.921	p2l:decl_cl:pl:#e-0lpl_interpr-e	l2p:elsmpx

And this is how the compounding is modeled with the iterative rule application approach. Changing the method just changes the rule ordering. The coverage specificity method will order the constraints by the increasing coverage. The condition specificity method will order the constraints by the decreasing condition specificity. Operationally, everything stays the same.

```
cxc.update_pred_params(method="yang-cvg")
cxc.gen_constr_df
```

c_id	linker	condition	spec_score	cvg_score	reg	confidence
p2l:decl_cl:pl:#e-0lpl_interpr-e	_+e_	[PL:e] & [sem:PL]	26.8	0.921	0.544	0.921
p2l:drv:prx_deverb-s	_+s_	[drv:PRX]	12.7	0.841	0.649	0.841
p2l:decl_cl:pl:#e-0ldef-0	_	[PL:e]	14.0	0.838	0.754	0.838
def-0	_	NaN	0.0	0.000	0.573	0.000

```
cxc.linker_scoreboard
```

linker	confidence	reg	c_id	removed_by
_+e_	0.921	0.544	p2l:decl_cl:pl:#e-0lpl_interpr-e	l2p:elsmpx
_+s_	0.841	0.649	p2l:drv:prx_deverb-s	
_	0.838	0.754	p2l:decl_cl:pl:#e-0ldef-0	

As explained earlier, the most confident linker is removed by a generative constraint, and the second most confident linker is returned instead.

```
cxc.y_pred
```

```
'_+s_'
```

```
cxc.update_pred_params(method="yang-cond")
cxc.gen_constr_df
```

c_id	linker	condition	spec_score	cvg_score	reg	confidence
p2l:decl_cl:pl:#e-0lpl_interpr-e	_+e_	[PL:e] & [sem:PL]	26.8	0.921	0.544	26.8
p2l:decl_cl:pl:#e-0ldef-0	_	[PL:e]	14.0	0.838	0.754	14.0
p2l:drv:prx_deverb-s	_+s_	[drv:PRX]	12.7	0.841	0.649	12.7
def-0	_	NaN	0.0	0.000	0.573	0.0

```
cxc.linker_scoreboard
```

linker	confidence	reg	c_id	removed_by
_+e_	26.8	0.544	p2l:decl_cl:pl:#e-0lpl_interpr-e	
_	14.0	0.754	p2l:decl_cl:pl:#e-0ldef-0	
_+s_	12.7	0.649	p2l:drv:prx_deverb-s	

```
cxc.y_pred
```

```
'_+e_'
```

Compounding Modeling: Results

We now turn to the actual results of compound modeling we obtained using the three conflict resolution methods outlined above. For each of the method, we collected and evaluated results with all possible combinations of the conflict resolution parameters discussed (the presence of the default constraints in the conflicts, constraint cooperation, linker elimination). Please note that, since the output of a "conflictless" (one-linker) resolution is deterministic, it is not informative of the models' performance. We therefore only run the models on the compounds with at least two conflicting linkers ($N=555$$N=555$).

Summary

We find that regularity is not a proper guide of compounding modeling. Because of their irregularity, it never prefers semantic constraints, which results in the absence of three pluralic linkers in the generations and a critical overproduction of another pluralic linker. We conclude that regularity is not a predictor itself but rather a meta-descriptor of constraint conflict resolution which is guided by other predictors.

Constraint specificity turns out to be a much better predictor. It is capable of producing all available linkers and does so with an adequate (for the first attempt) quality. We demonstrate that an empirical measure of specificity, such as constraint coverage, is prone to produce errors in specificity rankings due to its bind to a specific dataset. Differences in datasets lead to differences in rankings, which is unacceptable when modeling a language and not a dataset phenomenon. The formal way of assessing specificity based on the complexity of conditions of potential constraint applicability tackles this issue. Condition specificity produced constraint rankings very similar to those of coverage specificity, while resolving the incorrect rankings of the latter. This resulted in an identical or better F1 score for each linker. We conclude that condition specificity is a more appropriate predictor, which simultaneously is the only one to offer the advantage of robustness.

The dataset did not provide us with an adequate ground to test the three model parameters. The constraints conflicts in our nouns classes are constructed such that constraint cooperation is either impossible or supports all the generated constraints. Constraint cooperation thus does not make an impact on modeling in our data. The participation of the default linker did not change the results because it can only be useful with constraint cooperation on. Linker elimination never worked (except for one very narrow case) because only paradigmatic linkers or *-O-* were ever produced. We expect that the *-s-*\linked compounds would be the most fertile ground to test the three parameters. This linker is, however, largely underrepresented in our dataset so we do not make any conclusions about these parameters yet.

Below are the metrics of the three methods at their best parameters, and two baselines for comparison: a zero linker and a random baseline.

The good news is, all the three methods are considerably more accurate and linker-diverse than the baselines. This tells us that both regularity, coverage, and condition specificity have at least some predictive power.

The Albright & Hayes 2002 method is not very successful at modeling compounding. With an F1-score of only $0.24$$0.24$, it is actually closer performance-wise to the random baseline than to the specificity-based methods. The latter, in their turn, are not very far away from each other: the coverage specificity method achieves an F1-score of $0.48$$0.48$, condition specificity -- $0.53$$0.53$. These are quite high, considering the awfully imbalanced linker distribution in the dataset. Even despite the simplified constraint interaction in our dataset, we suppose that this is the great achievement for a mere initial attempt that only considers one single parameter to generate the linker.

One can be said right away: the specificity of the constraint -- both conditional and applicational -- is a much stronger predictor than its regularity (alone). Below, we examine the outputs for each method carefully.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

	acc	f1
yang-cond	0.712	0.526
yang-cvg	0.688	0.483
albright-hayes	0.638	0.242
rand	0.368	0.156
zero	0.485	0.093

Albright & Hayes 2002

The results are all the same for any model configuration.

We formulate the following questions with this regard:

1. What is the overall composition of the (best*) predictions?
2. How does the method's strategy of conflict resolution explain it?
3. Why are the same results produced regardless of the model configuration?

*The best configuration is marked in the metrics table with the `is_best_key` value.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

	acc	f1	is_best_key
+count_defl+coopl+l2p	0.638	0.242	True
+count_defl+coopl-l2p	0.638	0.242	False
+count_defl-coopl+l2p	0.638	0.242	False
+count_defl-coopl-l2p	0.638	0.242	False
-count_defl+coopl+l2p	0.638	0.242	False
-count_defl+coopl-l2p	0.638	0.242	False
-count_defl-coopl+l2p	0.638	0.242	False
-count_defl-coopl-l2p	0.638	0.242	False

The Overall Composition of The (Best) Predictions

The Albright & Hayes 2002 method (`AH02`) overproduces* *-O-* and underproduces* *-en-* approximately by half. Linkers *-"er-* and *-s-* are largely overproduced. All the other linkers are never produced.

*Relative to the true distribution of the linkers in the dataset.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

	reference	ours
_	0.485	0.813
_+en_	0.220	0.142
_+=er_	0.254	0.043
_+s_	0.020	0.002
_+e_	0.011	0.000
_+=e_	0.009	0.000
_+=_	0.002	0.000

This consequently leads to a zero accuracy and F1 score over the four non-produced linkers. Recall for *-"er-* is noticeably low, which, coupled with a suboptimal precision, leads to a low F1 score. Linker *-en-*, while underproduced and lacking recall, is still handled relatively well by the model; it's precision is very high, which results in an F1 score of $0.78$$0.78$. All the generations of *-s-* turn out to be incorrect. While having a great recall for a zero linker, `AH02` is not very precise over it.

Overall, the low F1 score is explained by underproduction (down to the absence) of most explicit linkers, on one hand, and by consequent overfavoring *-0-*, on the other.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

	acc	f1	N	prec	recall
avg	0.638	0.242	555.0	0.319	0.246
_	0.967	0.722	269.0	0.576	0.967
_+=er_	0.113	0.194	141.0	0.667	0.113
_+en_	0.639	0.776	122.0	0.987	0.639
_+s_	0.000	0.000	11.0	0.000	0.000
_+e_	0.000	0.000	6.0	0.000	0.000
_+=e_	0.000	0.000	5.0	0.000	0.000
_+=_	0.000	0.000	1.0	0.000	0.000

The overproduction of zero originates from the four non-produced linkers: they are all false predicted as zero. The underproduced linkers *-en-* and *-"er-* further contribute to overproduction; all but a few false predictions of the *-en-*\linked compounds (about $35\%$$35\%$) and all of the false predictions of the *-"er-*\linked compounds (almost $90\%$$90\%$) were predicted a zero linker. The explicit linkers, themselves, were false predicted instead of another linker just a few times.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

	_	_+=er_	_+en_	_+s_	_+e_	_+=e_	_+=_
_	0.967	0.030	0.004	0.000	0.0	0.0	0.0
_+=er_	0.887	0.113	0.000	0.000	0.0	0.0	0.0
_+en_	0.352	0.000	0.639	0.008	0.0	0.0	0.0
_+s_	1.000	0.000	0.000	0.000	0.0	0.0	0.0
_+e_	1.000	0.000	0.000	0.000	0.0	0.0	0.0
_+=e_	1.000	0.000	0.000	0.000	0.0	0.0	0.0
_+=_	1.000	0.000	0.000	0.000	0.0	0.0	0.0

For such a distribution of the predicted linkers to be the case, the constraint(s) for the zero linker must always win the ones for *-"0-*, *-s-*, *-"e-*, and *-e-*. They should also defeat the ones for *-en-* and *-"er-* in most cases. Let's investigate the corresponding constraint conflicts and their resolution under `AH02` to understand why this happens.

Closer Examination of Conflict Resolution

Below we can see all unique constraint conflicts.

-e-, *-"e-*, *-"0-*

We can clearly see right away why the three "semantics-only" linkers discussed above (*-e-*, *-"e-*, *-"0-*) are never produced. These linkers are always paradigmatic, which means the N1s that may attach them have a congruent plural marker. Apart from the optional (!) pluralic constraint that generates these linkers if such N1s have a plural reading, a morphological constraint is always applicable to them which generates *-0-* on the base of their plural form (e.g. "*-e-*\plural N1s mostly attach *-0-*."). While the pluralic constraints, themselves, are more regular than the default constraint, the ever present morphological constraints for these N1s are all more regular than their pluralic constraints. Thus, in a model based on regularity, these morphological constraints are preferred, and *-0-* is always produced. Since the pluralic constraints are the only ones that

may produce these linkers, there are no further constraints that would back them up to cooperatively overtake *-0-*.

Further constraints cannot win the competition with the morphological constraints, either. The prefixed deverbal constraint (*-s-*, e.g. N1 'Geschenk') is slightly less regular. The participation of the default constraint does not generally change a thing, either because only *-0-* is ever generated, anyways. The latter fact explains zero influence of linker elimination: there are none constraints to eliminate *-0-*.

Thus, in a regularity-based model, the "semantics-only" linkers *-e-*, *-"e-*, *-"0-* may never be produced. This is confirmed by the fact that the F1-score for these linkers in $0.0$$0.0$ in all configurations*.

-s-

Linker *-s-* bears a special, marginal character in our dataset. As discussed, our dataset mostly has simplex nouns, while *-s-* predominantly occurs in complex nouns. Also derivational and phonological constraints which are the primary source of this linker, are largely underused in our data. That leads to that the *-s-*\linked compounds in our dataset are not even nearly representative of its usual occurrence context. Specifically, we have a few mixed nouns ('Hemd') and a few simplex masculine and neuter nouns that occasionally attach *-s-* ('Hund', 'Kind'), which is, however, rather an exception. Consequently, *-s-* cannot be produced for these nouns: the mixed nouns constraint is a three-linker constraint, so *-s-* gets a tiny confidence, which is less than that of the default constraint, and the latter case ('Hund', 'Kind') does not have a further *-s-*\generating constraint defined over it (as said, this is more of an exception!).

The final picture is the same as with the "semantics-only" constraints. Neither linker elimination nor constraint cooperation nor the default constraint participation change the outcome: *-0-* is always produced. The F1-score for this linker in $0.0$$0.0$ in all configurations, too*.

Note that -s- is not representative over our dataset, at all. In the contexts where it has to appear, indeed, it is generated correctly. For example, after N1 'Nation' it is generated in all the configurations. This generation is incorrect, still, because in our dataset, it only appears in rather rare compounds such as 'Nationenwertung'. In other words, our dataset does not provide an adequate ground for -s- to be generated. The results over this linker are therefore to be ignored.

-en-

A more optimistic picture is observed with the *-en-*\linker. Except for rare cases, *-en-* appears paradigmatically in German. It can therefore appear either with a feminine noun or with a noun of the mixed declination. Considering feminine nouns, it appears often after schwa-final N1s; after consonant-final N1s, its appearance is mostly associated with a plural reading, while its presence in case of a singular reading is not necessary. Mixed nouns are very similar to consonant-final feminine nouns. `AH02` grasps all these patterns nicely. For schwa-final consonants, there are only the *-en-* constraint and the default constraint applicable; the former, being more regular, wins, and this holds for all the configurations. In consonant-final feminine nouns with plural reading and mixed nouns with plural reading, a morphological multi-linker constraint conflicts with a pluralic *-en-* constraint. The latter is regular for both classes and so it always wins the conflict. Even with constraint cooperation enabled, other linkers cannot compete with *-en-*. The morphological multi-linker constraint produces *-0-*, *-s-*, and *-en-* for both classes, so enabling constraint cooperation necessarily supports the winning *-en-*, too, and does not let it give up its position. Thus, *-en-* is always (correctly) produced for these three classes.

What concerns the remaining two classes (consonant-final feminine and mixed nouns with no plural interpretation), *-en-* can only be produced there with the morphological constraint. Since this constraint produces three linkers, the distributed confidence makes them incapable of competing. Only *-0-* is produced in these cases (and *-s-* just a few times, see N1 'Nation').

Similarly to the linkers above, none of the configurations differ in their results**. The three aforementioned classes are always predicted an *-en-*. Cooperation, as well as the default linker participation, cannot relegate *-en-* in favor of another linker; being a paradigmatic linker, it cannot be eliminated, either. In the latter two classes, the default participation linker can only bring harm to the chances of *-en-*, and constraint cooperation cannot save it because there is no other *-en-*\producing constraint to cooperate with. Since only *-0-* is produced, and it cannot be eliminated, linker elimination does not increase chances of *-en-* among such N1s.

-"er-

The distribution of the predictions for the *-"er-*\linker compounds is similar to what we saw over *-en-*. Only two linkers *-0-* and *-"er-* are ever generated for these nouns. On the morphological level, a two-linker constraint generates both *-0-* and *-"er-*, and consequently distributes the confidence among them. These constraints alone cannot determine the outcome. As apposed to the other pluralic linkers, there are two pluralic constraints defined over *-"er-*\plural N1s. They assign *-"er-* in case of a plural reading of N1, *-0-* otherwise. It is clear that these constraints are complementary, they cannot appear together and conflict. There is also a mass meaning constraint that also predicts *-0-* and is also complementary to those two. The confidence of the morphological constraint is distributed, and none of the two linkers predicted by it can win an ever present pluralic/mass constraint without cooperation. The default constraint is less regular than these constraints, too. Even when the

cooperation is on, the same thing happens as with *-en-*: by supporting one linker in cooperation, the morphological constraint inevitably supports the other by the same distributed confidence score, and so cooperation creates zero relative difference. Another constraint that predicts *-"er-* for animate nouns is too irregular, too, to make a difference. When cooperating with plural meaning, it either contributes to *-"er-* that would have won, anyways, for the reasons expounded above, or cannot even cooperatively beat the more regular singular reading constraint otherwise. The same happens with the default constraint: it either supports the already winning *-0-*, or cannot overtake the pluralic constraint, also not cooperatively. In other words, whichever constraint is the most regular alone, stays decisive regardless of the cooperation. Thus, the predicted linker is fully determined by the pluralic/mass meaning of N1. Indeed, *-0-* is always generated when the N1 has a mass or a singular meaning, and *-"er-* is generated otherwise. The only exception is when the N1 ends with a sibilant. In this case, a more regular sibilant-ending constraint generates *-0-*. In this case, the pluralic/mass constraint suffer the destiny of the constraints defeated by them otherwise. The mass or the singular meaning constraint only contributes to *-0-*, and the pluralic constraint cannot beat *-0-* (guess what!) even cooperatively. Needless to say, linker elimination cannot affect the result since neither *-"er-* (it is paradigmatic) either *-0-* can be removed. F1 scores confirm that the generations for the *-"er-* \linked compounds are immune to configuration changes**.

-0-

We can see from the summary, that *-0-*\linked compounds belong to all the same classes that were covered above. This is no wonder, since Schäfer & Pankratz 2018 purposefully selected N1s that can attach both pluralic and non-pluralic linkers. Thus, the predictions in these compounds are connected inalienably with the predictions of their counterparts with a pluralic linker. Since the model configuration does not change the outcome for the explicit linkers, it does not do so for *-0-*, either**.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

```
c_id
p2l:phon_fin:sibilant$-0                          _  0.970       0.970
p2l:decl_cl:pl:#er-0/er|!pl_interpr-0             _  0.622       0.622
def-0                                             _  0.573       0.573
p2l:decl_cl:pl:#er-0/er|def-0/er                  _  0.815       0.407
p2l:decl_cl:pl:#er-0/er|def-0/er             _+=er_  0.815       0.407


kind_bett:     _
                                           linker    reg  confidence
c_id
p2l:decl_cl:pl:#er-0/er|!pl_interpr-0             _  0.622       0.622
def-0                                             _  0.573       0.573
p2l:decl_cl:pl:#er-0/er|anim-er              _+=er_  0.501       0.501
p2l:decl_cl:pl:#er-0/er|def-0/er                  _  0.815       0.407
p2l:decl_cl:pl:#er-0/er|def-0/er             _+=er_  0.815       0.407


ohr_feige:     _
                                         linker    reg  confidence
c_id
def-0                                       _  0.573       0.573
p2l:decl_cl:mixed-0/s/en|def-0/s/en         _  0.996       0.332
p2l:decl_cl:mixed-0/s/en|def-0/s/en  _+en_  0.996       0.332
p2l:decl_cl:mixed-0/s/en|def-0/s/en   _+s_  0.996       0.332


schwester_modell:      _
                                              linker    reg  confidence
c_id
def-0                                            _  0.573       0.573
p2l:phon_fin:weak_F_cons$-0/s/en|def-0/s/en      _  1.000       0.333
p2l:phon_fin:weak_F_cons$-0/s/en|def-0/s/en  _+en_  1.000       0.333
p2l:phon_fin:weak_F_cons$-0/s/en|def-0/s/en   _+s_  1.000       0.333


vogel_zug:     _
                                          linker    reg  confidence
c_id
p2l:decl_cl:pl:#0_uml-0|def-0                  _  0.974       0.974
p2l:decl_cl:pl:#0_uml-0|pl_interpr-0_uml   _+=_  0.667       0.667
def-0                                          _  0.573       0.573


weg_gabelung:  _
                                    linker    reg  confidence
c_id
p2l:decl_cl:pl:#e-0|def-0                _  0.754       0.754
def-0                                    _  0.573       0.573
p2l:decl_cl:pl:#e-0|pl_interpr-e   _+e_  0.544       0.544


zahn_wechsel:  _
                                          linker    reg  confidence
c_id
p2l:decl_cl:pl:#e_uml-0|def-0                  _  0.716       0.716
def-0                                          _  0.573       0.573
p2l:decl_cl:pl:#e_uml-0|pl_interpr-e_uml  _+=e_  0.562       0.562
```

*The F1-score for the "semantics-only" linkers *-e-*, *-"e-*, *-"0-*, and for linker *-s-* is $0.0$$0.0$ in all configurations.

**The generations of *-en-*, *-"er-*, and *-0-* are robust with respect to configuration.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

	_+e_	_+=e_	_+=_	_+s_	_+en_	_+=er_	_
+count_defl+coopl+l2p	0.0	0.0	0.0	0.0	0.776	0.194	0.722
+count_defl+coopl-l2p	0.0	0.0	0.0	0.0	0.776	0.194	0.722
+count_defl-coopl+l2p	0.0	0.0	0.0	0.0	0.776	0.194	0.722
+count_defl-coopl-l2p	0.0	0.0	0.0	0.0	0.776	0.194	0.722
-count_defl+coopl+l2p	0.0	0.0	0.0	0.0	0.776	0.194	0.722
-count_defl+coopl-l2p	0.0	0.0	0.0	0.0	0.776	0.194	0.722
-count_defl-coopl+l2p	0.0	0.0	0.0	0.0	0.776	0.194	0.722
-count_defl-coopl-l2p	0.0	0.0	0.0	0.0	0.776	0.194	0.722

Summary

After scrutinizing for each linker how the `AH02` model handles the constraint conflicts in which they are (not) produced, we can now fully explain the model's generations.

1. What is the overall composition of the (best*) predictions?
2. How does the method's strategy of conflict resolution explain it?

The overall composition of the constraint conflicts provides a very unfertile ground for the regularity-based `AH02`. For the absolute majority (if not all) of the N1s in our dataset, there is some morphological *-O-*\generating constraint that is more regular than the others. Semantic constraints, which are the primary source of the explicit linkers (apart from *-s-*), are all less regular and cannot beat it. For many linkers, these are also the only source, and so they cannot cooperate to win because they are left alone on the constraint battlefield. In other cases, the morphological constraint generates multiple linkers. Supporting the explicit linker by cooperation, it only gives it a false hope: at the same time, this very constraint treacherously supports *-O-*, thus not letting it be relegated. Semantic constraints are therefore never supported individually. Thus, the most regular constraint always stays above regardless of the cooperation. Regularity leaves no chance for most of the explicit linkers to be generated. There is only a small group of noun classes where a constraint that generate an explicit linker, is the most regular one.

The most important conclusion is: the `AH02` model is unsuccessful because semantic constraints cannot work their way through to generate the pluralic linkers. Being more statistically more irregular, semantic constraints cannot win alone, and cooperation rarely (if ever) brings a relative advantage for them. For German nominal compounding, `AH02` is destined to continuously yield sad zeros.

3. Why are the same results produced regardless of the model configuration?

The fact discussed above that the most regular constraint cannot be relegated leads to identical generations with all the configurations. As explained, constraint cooperation either cannot support the semantic constraint, or does so alongside with the current leader; this way, constraint cooperation cannot change the generations. Being one of the hopeless irregular constraints, the default constraint cannot contribute to linker choice, either, similar to the sematic constraints. The generated linkers cannot be eliminated because they are either *-O-*, or paradigmatic (the few time *-s-* is generated, it is also not eliminated). Because of that, `AH02` configuration does not change the generations leading all the configurations to produce the best results*.

There was before one special case which depended on the configuration. It was the case of the unparadigmatic -en- case. The non-paradigmatic -en- constraint (now excluded) generated a number of false predictions of -en- in other classes (e.g. after N1 'Hund'), which led worse results. When both linker elimination and cooperation were disabled, the erroneous _-en-_ generation is unimpeded. When linker cooperation was back on, -en- could be defeated by -0- depending on whether the default constraint participated in the conflicts (then all false generations of -en- were hindered) or not (only some are then). If linker elimination was active, the non-paradigmatic -en- generation could not pass the restrictive paradicmatic -en- constraint. Now, since the responsible constraint is excluded from modeling, this case does not occur.

`AH02` demonstrates that regularity (alone) cannot guide linker generation. This is in line with the original results of Albright & Hayes 2002. In their experiment on modeling English past tense, they could only ever produce the regular past tense form with *-ed*. None of the irregular forms were generated. This makes us conclude that regularity only comprises the empirical result of the modeling by some other mysterious parameters. Itself, it cannot be used (alone) to model phenomena. It is not a predictor but a spectator.

Yang 2016: Coverage Specificity

The results are clustered by the metrics into two groups. Depending on whether linker elimination is enabled, F1-score reaches $0.37$$0.37$ or $0.38$$0.38$; accuracy is $0.7$$0.7$ in both cases with a negligible difference. Unexpectedly, linker elimination deteriorates performance.

With regard to that, a few questions arise:

1. What is the overall composition of the (best*) predictions?
2. How does the method's strategy of conflict resolution explain it?
3. Why does linker elimination deteriorates performance?
4. Why does not constraint cooperation and default constraint participation change anything?

*The best configuration is marked in the metrics table with the `is_best_key` value.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

	acc	f1	is_best_key
+count_defl+coopl-l2p	0.688	0.483	True
+count_defl-coopl-l2p	0.688	0.483	False
-count_defl+coopl-l2p	0.688	0.483	False
-count_defl-coopl-l2p	0.688	0.483	False
+count_defl+coopl+l2p	0.686	0.467	False
+count_defl-coopl+l2p	0.686	0.467	False
-count_defl+coopl+l2p	0.686	0.467	False
-count_defl-coopl+l2p	0.686	0.467	False

The Overall Composition of The (Best) Predictions

Contrary to `AH02`, the diversity of the generated linkers spans much wider: only *-"O-* is never produced by the coverage-based Yang 2016 method (`Y16-cvg`). `Y16-cvg` underproduces* *-en-* by exactly the same quantity. Linkers *-"er-* and *-"e-* are now underproduced* at a comparable, not critical, rate. The production of *-e-* and *-s-* is now comparable to the reference*. The increased share of the explicit linkers generated by `Y16-cvg` leads to a lower overproduction* of *-O-*.

*Relative to the true distribution of the linkers in the dataset.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

	reference	ours
_	0.485	0.643
_+=er_	0.254	0.159
_+en_	0.220	0.142
_+s_	0.020	0.029
_+e_	0.011	0.016
_+=e_	0.009	0.011
_+=_	0.002	0.000

Consequently, the F1 scores for the slightly underproduced *-en-*, *-"er-* are relatively high; even though they lack recall, their precision is respectable, producing an F1 score of $>0.5$$>0.5$ for both linkers. Linker *-"e-* now has a full recall and a very high precision, leading to an F1 score of $0.9$$0.9$. The generations of *-e-* are not very precise, plus it is missed in half of the cases, so its F1 score for *-e-* is low. The F1 score for *-"0-* is expectedly zero. As it turns out, all the generations of *-s-* are erroneous, which leads to a zero F1 score for this linker, too. Finally, similarly to `AH02`, the model achieves a high recall but a decreased precision for *-0-*.

Thus, the performance advancement relative to `AH02` is explained by the increased quality of predictions for the *-en-*, *-"er-*, and *-"e-* linkers, and the ability to produce the *-e-* linker. This is a great improvement of this model.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

	acc	f1	N	prec	recall
avg	0.688	0.483	555.0	0.503	0.493
_	0.866	0.744	269.0	0.653	0.866
_+=er_	0.447	0.550	141.0	0.716	0.447
_+en_	0.639	0.776	122.0	0.987	0.639
_+s_	0.000	0.000	11.0	0.000	0.000
_+e_	0.500	0.400	6.0	0.333	0.500
_+=e_	1.000	0.909	5.0	0.833	1.000
_+=_	0.000	0.000	1.0	0.000	0.000

All incorrect generations for linkers *-"er-* (slightly over a half), *-"0-* (all compounds), and almost all for *-en-* (about 35%35\%$) are counted towards *-0-*. For *-e-*, about two third of incorrect generations are *-s-*. The linker *-s-* itself is taken up entirely by false generations of *-0-* but mostly *-"er-*. The incorrect predictions for the zero-linked compounds are distributed among all the explicit linkers except for *-"0-*. In sum, the confusion tendency of `Y16-cvg` is similar to that of `AH02`: *-0-* is still the default fallback when an explicit linker is predicted incorrectly.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

	_	_+=er_	_+en_	_+s_	_+e_	_+=e_	_+=_
_	0.866	0.056	0.004	0.048	0.022	0.004	0.0
_+=er_	0.553	0.447	0.000	0.000	0.000	0.000	0.0
_+en_	0.352	0.000	0.639	0.008	0.000	0.000	0.0
_+s_	0.091	0.909	0.000	0.000	0.000	0.000	0.0
_+e_	0.167	0.000	0.000	0.333	0.500	0.000	0.0
_+=e_	0.000	0.000	0.000	0.000	0.000	1.000	0.0
_+=_	1.000	0.000	0.000	0.000	0.000	0.000	0.0

To form such a distribution, the constraint(s) for the zero linker must win and lose the ones for *-en-*, *-"er-*, *-"e-*, and *-e-* in an alternating fashion. It should also always win the ones for *-"0-*. Let's investigate the corresponding constraint conflicts and their resolution under `Y16-cvg` to understand why this happens.

Closer Examination of Conflict Resolution

Below we can see all unique constraint conflicts. In the analysis of the `AH02` results, we extensively described and characterized all the unique conflicts for all linkers. We will therefore proceed directly to their resolution with `Y16-cvg` now.

-"0-

There is a single pluralic constraint that produces *-"0-*. It has a higher coverage than the single morphological constraint that produces *-0-*, so *-"0-* is never produced. Constraint cooperation is irrelevant since the *-"0-*\generating linker comes alone. Since only *-0-* is produced, both the default constraint participation and linker elimination do not contribute. The F1 score for

-"0- is $0.0$$0.0$ for all configurations*.

NB! Please note that the coverages for the semantic and the morphological constraints come from two different datasets. The coverage of the semantic constraint is calculated over a derivative of Schäfer & Pankratz 2018; it was originally designed to investigate pluralic linkers and has an artificially increased share of compounds with those. If the coverages were measured over the same dataset, the coverage of the semantic constraint would have inevitably been lower because the nouns targeted by it are a subset of the nouns targeted by the morphological constraint. This case illustrates a big disadvantage of the coverage-based model: bind to the specific dataset.

-e-

Three linkers are generated for the *-e-*\linked compounds. Following are the constraints that generated them, in an order of decreasing confidence (increasing coverage). Linker *-e-* is produced by a pluralic semantic constraint. The sibilant constraint assigns *-0-*. A rare derivational constraint generates *-s-*, closely followed by the morphological *-e-*\plural constraint that generates *-0-*, too. Finally, there is the default constraint.

Identical to *-"0-*, both *-e-* and *-s-* are generated by a single constraint each, and cannot be supported by any other constraint. The default constraint has a coverage score of $0.0$$0.0$ and is therefore virtually nonexistent for the model°: alone, it cannot win any other constraint, and its cooperation brings literally zero. Given the confidence order of the remaining constraints, we can see that there is a single case where constraint cooperation can be decisive: if a sibilant-final N1s will have a plural meaning, it will be predicted a *-e-* without and *-0-* with (sibilant + morphological) constraint cooperation. This case, however, does not occur in our dataset. From what we have left, either *-0-* is on the top by the sibilant constraint and it cannot be overthrown by the explicit linkers, or *-0-* is on the top by the morphological constraint with none generated explicit linkers, or an explicit linker is on the top, and then the less confident morphological *-0-* constraint can only be supported by the default constraint, which, as already pointed out, has zero effect. Thus, we have a situation identical to *-en-* and *-"er-* with `AH02`: whichever constraint has the highest confidence cannot be beaten by any other constraint regardless of constraint cooperation or default constraint participation.

We have left a single means to affect the outcome of the modeling for this linker, and it does affect it here, indeed. There is a restrictive constraint that prohibits *-e-* for derived N1s. If linker elimination is on, an N1 with a plural meaning which should have *-e-* by the pluralic constraint, will be deprived of it if its not simplex. This is best demonstrated by the example of compound 'Geschenkeliste'. When no linker elimination is not active, the correct *-e-* is produced by the pluralic constraint. With linker elimination, *-e-* is removed by the restrictive constraint, and an incorrect *-s-* is generated instead°°. The harmful influence of the restrictive constraints for this linker is confirmed by the F1 scores; they are split onto two clusters. If linker elimination is off, the scores are higher**.

-en-

The resolution of the *-en-*\linker conflicts happens to be identical to that by `AH02`. In case of a schwa-final N1, the schwa *-en-* constraint competes with the "invisible" (see above) default constraint and always wins. In other cases, the morphological constraints are multi-linker constraints and are indecisive as such. It then all comes down to the plural semantics of N1. Beside the highest regularity, the pluralic *-en-* constraints happen to have the least coverage which brings them to the top in `Y16-cvg`, too. This way, *-en-* is generated when N1 has a plural meaning, *-0-* (rarely *-s-*, e.g. 'Nation') otherwise. Why all the configurations produce the same results***, is explained for `AH02`.

-"er-

The resolution of the *-"er-*\linker conflicts happens to be almost identical to that by `AH02`, too. There is a two-linker morphological constraint, the sibilant constraint, and a few further semantic constraints that predict either *-0-*, or *-"er-*. Coverage creates almost the same constraint order that `AH02` had: the sibilant constraint has a lower coverage than the semantic constraints, which, in their term, have a lower coverage than the morphological constraint. This leads to identical constraint resolution process; then, identically, neither constraint cooperation nor further parameters change the generations***, see the explanation in `AH02`.

The only difference in constraint order is that the animacy *-"er-* constraint has the least coverage, hence, the highest confidence in `Y16-cvg`. When the N1 designates an animal or a person, this constraint becomes first and, for the reasons explained in `AH02`, cannot be overthrown by *-0-*. Then, on the contrary to `AH02` which predicted an incorrect *-0-* for these nouns, `Y16-cvg` predict the correct *-"er-* for them. This explains better results of this model for *-"er-*.

NB! Again, the sibilant and the pluralic constraints were measured over different datasets. Over one dataset, the coverages, hence, generations, might have been different.

-"e-

Same as *-"0-* but the single pluralic constraint has a lower coverage than the morphological constraint (as it should). The

model predicts *-"e-* when N1 has a plural meaning, *-O-* otherwise. Configuration does not change anything*** for the same reasons as with *-"O-*.

-s-

See explanation in `AH02` about the unrepresentative composition of the *-s-*\linked compounds in our dataset and a consequent inability of the model to produce *-s-* for them. Since all the generations are incorrect, the F1 score is $0.0$$0.0$ for all configurations*.

-O-

Since all the generations for the explicit linkers are robust to configuration changes, so are the generation for *-O-*, see `AH02` for details. The only changing *-e-* case produces *-s-* when incorrect, so it does not affect the scores for *-O-*, either***.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

```
==== LINKER _+=_ ====

 mutter_+=_treff:       _
                                          linker  cvg_score  confidence
 c_id
 p2l:decl_cl:pl:#0_uml-0|def-0                 _      0.993       0.993
 p2l:decl_cl:pl:#0_uml-0|pl_interpr-0_uml   _+=_      0.961       0.961
 def-0                                         _      0.000       0.000


==== LINKER _+e_ ====

geräusch_+e_macher:    _
                           linker  cvg_score  confidence
c_id
p2l:phon_fin:sibilant$-0        _      0.915       0.915
p2l:drv:prx_deverb-s         _+s_      0.841       0.841
p2l:decl_cl:pl:#e-0|def-0       _      0.838       0.838
def-0                           _      0.000       0.000


geschenk_+e_liste:     _+e_
                                 linker  cvg_score  confidence
c_id
p2l:decl_cl:pl:#e-0|pl_interpr-e   _+e_      0.921       0.921
p2l:drv:prx_deverb-s               _+s_      0.841       0.841
p2l:decl_cl:pl:#e-0|def-0             _      0.838       0.838
def-0                                 _      0.000       0.000


geschenk_+e_sack:      _+s_
                           linker  cvg_score  confidence
c_id
p2l:drv:prx_deverb-s         _+s_      0.841       0.841
p2l:decl_cl:pl:#e-0|def-0       _      0.838       0.838
def-0                           _      0.000       0.000


weg_+e_netz:   _+e_
                                  linker  cvg_score  confidence
c_id
p2l:decl_cl:pl:#e-0|pl_interpr-e   _+e_      0.921       0.921
p2l:decl_cl:pl:#e-0|def-0             _      0.838       0.838
def-0                                 _      0.000       0.000


==== LINKER _+en_ ====

bett_+en_zahl: _+en_
                                        linker  cvg_score  confidence
c_id
p2l:decl_cl:mixed-0/s/en|pl_interpr-en  _+en_      0.961       0.961
p2l:decl_cl:mixed-0/s/en|def-0/s/en        _       0.987       0.329
p2l:decl_cl:mixed-0/s/en|def-0/s/en    _+en_       0.987       0.329
p2l:decl_cl:mixed-0/s/en|def-0/s/en     _+s_       0.987       0.329
def-0                                      _      0.000       0.000


nation_+en_wertung:    _+s_
```

*The F1 scores for *-"0-* and *-s-* are $0.0$$0.0$ for all configurations.

**When linker elimination is off, the F1 scores are higher for the *-e-*\linker.

***All the configurations produce the same results for *-en-*, *-"er-*, *-"e-*, and *-0-*.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

	_+=_	_+e_	_+en_	_+=er_	_+=e_	_+s_	_
+count_defl+coopl+l2p	0.0	0.286	0.776	0.55	0.909	0.0	0.744
+count_defl+coopl-l2p	0.0	0.400	0.776	0.55	0.909	0.0	0.744
+count_defl-coopl+l2p	0.0	0.286	0.776	0.55	0.909	0.0	0.744
+count_defl-coopl-l2p	0.0	0.400	0.776	0.55	0.909	0.0	0.744
-count_defl+coopl+l2p	0.0	0.286	0.776	0.55	0.909	0.0	0.744
-count_defl+coopl-l2p	0.0	0.400	0.776	0.55	0.909	0.0	0.744
-count_defl-coopl+l2p	0.0	0.286	0.776	0.55	0.909	0.0	0.744
-count_defl-coopl-l2p	0.0	0.400	0.776	0.55	0.909	0.0	0.744

°The zero confidence of the default linker does not make undefined the generations produced by multi-linker constraints only. For example, mixed nouns are predicted a *-0-*, *-en-*, or a *-s-* with the same confidence. While the confidences are identical, the auxiliary property (regularity for `Y16–cvg`) is higher for *-0-* after inclusion of the default constraint. Even in case when the default constraint does not participate in constraint conflict, the three linkers are sorted in this exact order, and the first, or *-0-*, is always taken. Thus, *-0-* is supported by default by the constraint resolution mechanism itself, exactly as it would be if the default constraint could contribute itself.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

linker	confidence	reg	c_id
_	0.329	1.569	def-0, p2l:decl_cl:mixed-0/s/enldef-0/s/en
_+en_	0.329	0.996	p2l:decl_cl:mixed-0/s/enldef-0/s/en
_+s_	0.329	0.996	p2l:decl_cl:mixed-0/s/enldef-0/s/en

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

linker	confidence	reg	c_id
_	0.329	0.996	p2l:decl_cl:mixed-0/s/enldef-0/s/en
_+en_	0.329	0.996	p2l:decl_cl:mixed-0/s/enldef-0/s/en
_+s_	0.329	0.996	p2l:decl_cl:mixed-0/s/enldef-0/s/en

°°When linker elimination is on, the restrictive simplex *-e-* constraint prohibits the correct *-e-* in derived N1s with a plural interpretation.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

linker	confidence	reg	c_id	removed_by
_+e_	0.921	0.544	p2l:decl_cl:pl:#e-0lpl_interpr-e	l2p:elsmpx
_+s_	0.841	0.649	p2l:drv:prx_deverb-s	
_	0.838	1.327	def-0, p2l:decl_cl:pl:#e-0ldef-0	

Summary

After scrutinizing for each linker how the `Y16-cvg` model handles the constraint conflicts in which they are (not) produced, we can now fully explain the model's generations.

1. What is the overall composition of the (best*) predictions?
2. How does the method's strategy of conflict resolution explain it?

Coverage specificity defeats the main disadvantage of `AH02` and gives a way for the semantic constraints to work through and generate the pluralic linkers. Being quite coverage-specific, the pluralic constraints beat most of the *-O-* generating constraints, so N1s with a plural meaning are consistently predicted a pluralic linker with `Y16-cvg`. Otherwise, *-O-* is mostly predicted, similar to `AH02`.

3. Why does linker elimination deteriorates performance?

There is a single case when linker elimination affects the generations within `Y16-cvg`: if a derived *-e-*\plural N1 has a plural reading, it is generated an *-e-*, which can be hindered by a restrictive constraint that prohibits *-e-* after derived nouns. In our dataset, it is exemplified by N1 'Geschenk'. Since our dataset has a number of entries where 'Geschenk' attaches *-e-*, indeed, linker elimination replaces the correct *-e-* generation with an incorrect *-s-*.

This single case makes it look like the overall performance of the model drops with enabling linker elimination. However, linker elimination does not change the prediction for all other noun classes. We deem this case too narrow to make any conclusions about the role of linker elimination in general.

4. Why does not constraint cooperation and default constraint participation change anything?

The default constraint has a coverage score of $0.0$$0.0$ which makes it "invisible" for the model: it cannot win any constraint itself, and cannot contribute anything within constraint cooperation. This explains why the default constraint participation does not change the predictions.

What concerns constraint cooperation, the very composition of the conflicts makes the most confident constraint immune to the attacks, similar to `AH02`. There is always a background morphological constraint for most of the noun classes in our dataset. It either generates *-O-* only, or multiple linkers necessarily including *-O-* and the pluralic linker. In our dataset, it happens so that when there is a single morphological constraint, there is also only a single semantic constraint in the competition, and when there is a multi-linker constraint, it generates all the same linkers that the other constraint jointly. In the first case, constraint cooperation is irrelevant, in the second -- the multi-linker constraint inevitably supports the most confident constraint because it generates one of its linkers. Because of that, if a constraint becomes first, the less confident constraints are either alone and cannot cooperate (remember, the default linker is "invisible"!), or they are multi-linker constraints that support the current first linker, as well. Of course, this is not a universal feature for all German compounds; we can easily construct compounds for which constraint cooperation would affect the generation. They are just not present in our dataset, so it may be not best suitable to test it. Our intuition is, this parameter could be crucial for *-s-*\linked compounds. Same goes for linker elimination: it should be clearly visible on the *-s-* linker but not on our *-s-*\poor dataset, see `AH02`.

`Y16-cvg` suggests that specificity may be a more appropriate guide of linker generation. When applying more specific constraints first, we leave a chance for the "peripheral" constraints to have a say. Belonging to such peripheral constraints, semantic constraints finally find their chance to produce pluralic linkers, some of which cannot be produced by any other constraints. `Y16-cvg` does not let the low frequency of the marginal linkers become a punishment for them. Right opposite, it treats them special and helps them come through the overwhelming statistical presence of *-O-*. This way, `Y16-cvg` reduces the underrepresentation problem of `AH02`.

At the same time, the example of *-"O-* (and partially *-"er-*) demonstrates how binding to statistical properties of a certain dataset may be harmful for modeling an abstract linguistic phenomenon. Thus, just because the two different datasets had different shares of *-"O-*\plural N1s, this linker could not be generated. Coverage is a property of the dataset and not language, so even small differences in the data may lead to discrepancies of the model generation fit on different dataset, which is very much undesirable. A good model should be able to abstract away from specific numbers and should be able to capture the general language and not dataset pattern.

Yang 2016: Condition Specificity

Similar to `AH02`, the results are identical same across the configurations.

We then formulate the same questions to address:

1. What is the overall composition of the (best*) predictions?
2. How does the method's strategy of conflict resolution explain it?
3. Why are the same results produced regardless of the model configuration?

*The best configuration is marked in the metrics table with the `is_best_key` value.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

	acc	f1	is_best_key
+count_defl+coopl+l2p	0.712	0.526	True
+count_defl+coopl-l2p	0.712	0.526	False
+count_defl-coopl+l2p	0.712	0.526	False
+count_defl-coopl-l2p	0.712	0.526	False
-count_defl+coopl+l2p	0.712	0.526	False
-count_defl+coopl-l2p	0.712	0.526	False
-count_defl-coopl+l2p	0.712	0.526	False
-count_defl-coopl-l2p	0.712	0.526	False

The Overall Composition of The (Best) Predictions

The condition-based Yang 2026 method (`Y16-cond`) finally produces all the linkers. It generates the same amount of linkers *-en-*, *-e-*, and *-"e-* as `Y16-cvg`, produces *-"er-* slightly more and finally produces *-"O-*, even overly productive compared to the reference*. The production of *-s-* drops significantly, however. The differences in production cancel each other out, so the share of *-O-* is almost the same in `Y16-cvg` and `Y16-cond`.

*Relative to the true distribution of the linkers in the dataset.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

	reference	ours
_	0.485	0.645
_+=er_	0.254	0.171
_+en_	0.220	0.142
_+e_	0.011	0.016
_+=_	0.002	0.013
_+=e_	0.009	0.011
_+s_	0.020	0.002

The metrics of `Y16-cond` are similar to those of `Y16-cvg`. The F1 scores for the slightly underproduced *-en-* and *-"er-* are relatively high again; even though they lack recall, their precision is respectable, producing an F1 score of $>0.5$$>0.5$ for both. Linker *-"e-* has a full recall and a very high precision, gaining as much as $0.9$$0.9$ of F1 score. Linkers *-e-* and *-"O-* have a full recall, too; but their generations are not very precise, so the F1 score for these two are low. Just like with the other two models, all the generations of *-s-* are all erroneous, which leads to a zero F1 score for this linker. Finally, similarly to the two models, `Y16-cond` achieves a high recall but a decreased precision for *-O-*.

Thus, the performance advancement relative to `Y16-cvg` is explained by the increased recall of *-"er-* and ability to generate *-"O-*. Being able to produce all linkers at least not worse than the previous models make it our best-performing model not only quantitatively but also qualitatively.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

	acc	f1	N	prec	recall
avg	0.712	0.526	555.0	0.527	0.645
_	0.888	0.762	269.0	0.668	0.888
_+=er_	0.489	0.585	141.0	0.726	0.489
_+en_	0.639	0.776	122.0	0.987	0.639
_+s_	0.000	0.000	11.0	0.000	0.000
_+e_	0.500	0.400	6.0	0.333	0.500
_+=e_	1.000	0.909	5.0	0.833	1.000
_+=_	1.000	0.250	1.0	0.143	1.000

There are no noteworthy differences of `Y16-cond`'s confusion matrix relative to `Y16-cvg`. This confusion matrix finally confirms the fact that *-0-* is the default fallback when an explicit linker is predicted incorrectly by our iterative-rule models.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

	_	_+=er_	_+en_	_+s_	_+e_	_+=e_	_+=_
_	0.888	0.059	0.004	0.000	0.022	0.004	0.022
_+=er_	0.511	0.489	0.000	0.000	0.000	0.000	0.000
_+en_	0.352	0.000	0.639	0.008	0.000	0.000	0.000
_+s_	0.091	0.909	0.000	0.000	0.000	0.000	0.000
_+e_	0.500	0.000	0.000	0.000	0.500	0.000	0.000
_+=e_	0.000	0.000	0.000	0.000	0.000	1.000	0.000
_+=_	0.000	0.000	0.000	0.000	0.000	0.000	1.000

To form such a distribution, the constraint(s) for the zero linker must win and lose the ones for *-en-*, *-"er-*, and *-"e-* in an alternating fashion. It should also win the ones for *-"0-* and *-e-* in most cases. Let's investigate the corresponding constraint conflicts and their resolution under `Y16-cond` to understand why this happens.

Closer Examination of Conflict Resolution

Conflict resolution by `Y16-cond` is very similar to that by `Y16-cvg`. After all, both models are specificity based, they just define specificity as a formal or empirical measure, respectively. Given this similarity, we will not dive too much into `Y16-cond`. We will highlight its unique peculiarities, and will refer the reading to `Y16-cvg`, otherwise.

-"0-

Being more specific, the pluralic constraint defeats the morphological constraint in N1s with a plural meaning. Thus, `Y-cond` defeats the harmful bind of `Y-cvg` to the empirical N1 distribution in a specific dataset.

Identically to `Y-cvg`, the default constraint, having a condition specificity of zero, has literally no impact on the modeling outcome. Further parameters do not affect the generation* for exactly the same reasons as with `Y-cvg`.

-e-

Three linkers are generated for the *-e-*\linked compounds. Following are the constraints that generated them, in an order of decreasing confidence (increasing coverage). Linker *-e-* is produced by a pluralic semantic constraint. The sibilant constraint assigns *-0-*. A rare derivational constraint generates *-s-*, closely followed by the morphological *-e-*\plural constraint that generates *-0-*, too. Finally, there is the ("invisible") default constraint.

The constraints that generate the linkers in the *-e-*\linked compounds are ordered by condition specificity in the exact same order as in `Y16-cvg` except for the morphological *-0-* constraint that is now more confident than the derivational constraint. The only noun class that is affected by this change is derived N1s ('Geschenk') without a plural reading. They were predicted *-s-* in `Y16-cvg` and are now predicted *-0-*; both are incorrect. All the other cases are resolved the same as `Y16-cvg`, so refer to it for details.

What is different in `Y-cond` is that the restrictive derived N1 *-e-* constraint is now not confident enough to remove *-e-* in cases like 'Geschenkeliste'. Consequently, the F1 score of our model for *-e-* the same when linker elimination is on or off. For other parameters, see `Y-cvg`.

-en-

Identical to `Y16-cvg`. The decisive (non-morphological) constraints (pluralic constraints and the schwa constraint) that had higher coverage specificity with `Y16-cvg` have higher condition specificity, too. The only difference is for the 'Nation' type: the *-ion* constraint is less specific than the multi-linker morphological constraint, even after distributing the score between three linkers. The prediction is then *-0-* or *-s-* depending on constraint cooperation; both are incorrect though.

-"er-

Identical to `Y16-cvg`. The decisive (non-morphological) constraints (pluralic and further semantic constraints, the sibilant constraint) that had higher coverage specificity with `Y16-cvg` have higher condition specificity, too. The only difference is that the pluralic *-"er-* constraint is now more confident than the sibilant constraint. Because of that, sibilant-final N1s with a plural meaning are predicted an *-"er-* instead of *-0-*. This small noun class is what makes the prediction of `Y16-cond` over *-"er-* slightly better than with `Y16-cvg`.

-"e-

Identical to `Y16-cvg`: the pluralic constraint is more specific.

-s-

See explanation in `AH02` about the unrepresentative composition of the *-s-*\linked compounds in our dataset and a consequent inability of the model to produce *-s-* for them. Since all the generations are incorrect, the F1 score is $0.0$$0.0$ for all configurations**.

-0-

Since all the generations for the explicit linkers are robust to configuration changes, so are the generation for *-0-**, see `AH02` for details.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

```
c_id
p2l:decl_cl:pl:#er-0/er|!pl_interpr-0       _    [PL:er] & [sem:!PL]        25.2        25.2
p2l:phon_fin:sibilant$-0                    _  [coda:S] | [coda:CLT]        12.8        12.8
p2l:decl_cl:pl:#er-0/er|def-0/er            _                [PL:er]        14.0         7.0
p2l:decl_cl:pl:#er-0/er|def-0/er       _+=er_                [PL:er]        14.0         7.0
def-0                                       _                   NaN         0.0         0.0


kind_bett:     _+=er_
                                       linker                           condition  spec_score  confid
c_id
p2l:decl_cl:pl:#er-0/er|anim-er        _+=er_  [PL:er] & [[sem:anim] | [sem:pers]]        28.0
p2l:decl_cl:pl:#er-0/er|!pl_interpr-0       _                  [PL:er] & [sem:!PL]        25.2
p2l:decl_cl:pl:#er-0/er|def-0/er            _                              [PL:er]        14.0
p2l:decl_cl:pl:#er-0/er|def-0/er       _+=er_                              [PL:er]        14.0
def-0                                       _                                 NaN         0.0


ohr_feige:     _
                                    linker condition  spec_score  confidence
c_id
p2l:decl_cl:mixed-0/s/en|def-0/s/en      _    [PL:m]        14.0       4.667
p2l:decl_cl:mixed-0/s/en|def-0/s/en  _+en_    [PL:m]        14.0       4.667
p2l:decl_cl:mixed-0/s/en|def-0/s/en   _+s_    [PL:m]        14.0       4.667
def-0                                    _       NaN         0.0       0.000


schwester_modell:      _
                                                 linker                    condition  spec_score  confidenc
c_id
p2l:phon_fin:weak_F_cons$-0/s/en|def-0/s/en      _  [G:F] & [PL:en] & [coda:CN]        39.7      13.23
p2l:phon_fin:weak_F_cons$-0/s/en|def-0/s/en  _+en_  [G:F] & [PL:en] & [coda:CN]        39.7      13.23
p2l:phon_fin:weak_F_cons$-0/s/en|def-0/s/en   _+s_  [G:F] & [PL:en] & [coda:CN]        39.7      13.23
def-0                                            _                         NaN         0.0       0.00


vogel_zug:     _+=_
                                          linker           condition  spec_score  confidence
c_id
p2l:decl_cl:pl:#0_uml-0|pl_interpr-0_uml  _+=_  [PL:uo] & [sem:PL]        26.8        26.8
p2l:decl_cl:pl:#0_uml-0|def-0                _              [PL:uo]        14.0        14.0
def-0                                        _                 NaN         0.0         0.0


weg_gabelung:  _+e_
                                   linker          condition  spec_score  confidence
c_id
p2l:decl_cl:pl:#e-0|pl_interpr-e   _+e_  [PL:e] & [sem:PL]        26.8        26.8
p2l:decl_cl:pl:#e-0|def-0             _             [PL:e]        14.0        14.0
def-0                                 _                NaN         0.0         0.0


zahn_wechsel:  _+=e_
                                          linker           condition  spec_score  confidence
c_id
p2l:decl_cl:pl:#e_uml-0|pl_interpr-e_uml  _+=e_  [PL:ue] & [sem:PL]        26.8        26.8
p2l:decl_cl:pl:#e_uml-0|def-0                _              [PL:ue]        14.0        14.0
def-0                                        _                 NaN         0.0         0.0
```

*The F1 scores are identical regardless of the configuration.

**The F1 score for *-s-* is always $0.0$$0.0$.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

	_+=_	_+e_	_+en_	_+=er_	_+=e_	_+s_	_
+count_defl+coopl+l2p	0.25	0.4	0.776	0.585	0.909	0.0	0.762
+count_defl+coopl-l2p	0.25	0.4	0.776	0.585	0.909	0.0	0.762
+count_defl-coopl+l2p	0.25	0.4	0.776	0.585	0.909	0.0	0.761
+count_defl-coopl-l2p	0.25	0.4	0.776	0.585	0.909	0.0	0.761
-count_defl+coopl+l2p	0.25	0.4	0.776	0.585	0.909	0.0	0.762
-count_defl+coopl-l2p	0.25	0.4	0.776	0.585	0.909	0.0	0.762
-count_defl-coopl+l2p	0.25	0.4	0.776	0.585	0.909	0.0	0.761
-count_defl-coopl-l2p	0.25	0.4	0.776	0.585	0.909	0.0	0.761

Summary

The purpose of the `Y-cond` model was to abstract the empirical specificity of a model described by its coverage to a kind of formal specificity. The goal of this abstraction was to preserve the valuable information stored by the coverage score (as demonstrated by `Y-cvg`) while eliminating the harmful connection of the scores to the N1 distribution of a specific dataset. After all, we model a language phenomenon and not a dataset, and we want our specificity score to be robust to the data.

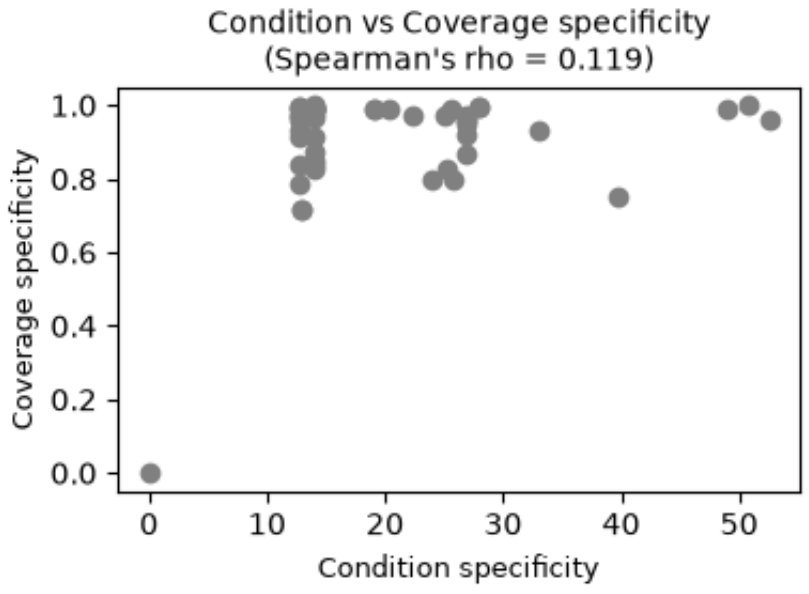
We find that `Y16-cond` handles this objective very successfully. The overall correlation between the coverage score and the condition specificity score of the constraint is very low, indeed: the Spearman's correlation coefficient is only $0.12$$0.12$*. If we look at the scatter plot of the two variables, we will see the reason clearly. There are many constraints with a vast range of coverage that are assigned the same condition specificity. For example, conditions `[PL:uo]` (*-"O*-\plural form) and `[coda:e]` (schwa-final nouns), assigned the same condition specificity score $14$$14$, have an item coverage of $0.007$$0.007$ and $0.171$$0.171$, respectively -- this is an enormous difference! But here is the catch: schwa-final nouns may not form a plural form with *-"O*. What this example demonstrates is that the condition specificity score is not meant to provide an overall ranking of all constraints that would be faithful to their empirical applicability span. The low overall correlation is just not informative given numerous instances of incompatibility between many constraints. Conversely, condition specificity meant to estimate the formal complexity of overlapping conditions which are often in conflict, and thus abstract away from their coverage statistics over certain datasets. For example, we believe, no one would argue that condition `[PL:uo]` (*-"O*-\plural form) is less specific than `[PL:uo] & [sem:PL]` (*-"O*-\plural form & plural meaning). `Y16-cvg` demonstrated how this clear specificity order can be corrupted by measurements over different datasets. In response to that, `Y16-cond` demonstrated how the formal specificity estimation can tackle this problem. If we consider the rankings of the actually overlapping (conflicting) generative constraints in each compound from the two models, we realize that the average Kendall's tau-b rank correlation score of $0.75$$0.75$**. That is, the ordering of the actually conflicting constraints is very similar between the two models. This attests that condition specificity is capable of comprising the empirical coverage information in it. On the other hand, the F1 score of `Y16-cond` is better or at least identical to `Y16-cvg` for every linker***. The examples of *-"O-* and *-"er-* demonstrate that the improvement originated from a more adequate constraint specificity ordering relative to what the coverage score provided. We therefore conclude that condition specificity is a more suitable measure of the three to model German compounding. `Y16-cond` is with that our best performing and the only robust model.

*The overall correlation between the coverage and the condition specificity score is poor: Spearman's rho is as low as $0.12$$0.12$. We choose Spearman's rho because both distributions are non-normal as indicated by Shapiro test.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

```
Shapiro, null hypothesis: normal distribution
Coverage Specificity:  0.0
Condition Specificity:  0.0
```

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.



**The average Kendall's tau-b score over all compounds is $0.75$$0.75$ between two rankings (range of possible values: $[-1; 1]$$[-1; 1]$).

We are aware that the rankings of 2-4 constraints have a limited statistical power. However, we employed the coefficient in order to calculate the empirical correlation between the rankings and not to confirm a statistical association between them.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

```
0.749
```

***The F1 score of `Y16-cond` is better or at least identical to `Y16-cvg` for every linker.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

	cvg	cond	cond ? cvg
avg	0.483	0.526	>
_	0.744	0.762	>
_+=er_	0.550	0.585	>
_+en_	0.776	0.776	=
_+s_	0.000	0.000	=
_+e_	0.400	0.400	=
_+=e_	0.909	0.909	=
_+=_	0.000	0.250	>

LLM Experiments

GLM-5.3-Flash: the latest model from Z.ai released at the end of August 2026 called GLM-5.3-Flash. This is a multilingual multimodal 320B model with 18B active parameters.

The HF model card: https://huggingface.co/zai-org/GLM-5.3-Flash. GLM Cloud Console: https://console.gmicloud.ai/user-console/ie/model-hub/llm/60e50c31-b7d0-45a9-b83b-905a0d169c10. Technical report: https://arxiv.org/abs/2602.15763.

At the state of Sep 21, 2026:

This model is the best performing open-source model on the German benchmark DACH-bench spanning six evaluation tasks on different aspects of NLU: https://dach.peerbench.ai, model profile https://dach.peerbench.ai/models/z-ai/glm-5.3-flash, being the 8th model overall. It outperforms all QWEN's, all Gemini Flash versions until 3.7, and GPT 5.6-Terra.

It is also the best open-source model on a more extensive EuroEval benchmark for German including the six DACH-bench but also further NLU tasks: https://huggingface.co/spaces/EuroEval/euroeval_leaderboard, being the 2nd model overall for German there. It is comparable to Gemini-3.7-Flash on this benchmark and is outperformed only by GPT-6-Astra.

Most importantly for our use case, this model is the best open-source model on the ScaLa dataset: https://dach.peerbench.ai/tab-scala_de. ScaLa-de tests the LLMs for grammatical acceptability of German sentences on grammatically correct vs. minimally (!) corrupt German sentences. For example, the model correctly marks the sentence 'Ob dieser Satz korrekt ist, weißt er nicht.' ungrammatical but '... weiß er nicht.' grammatically correct. That indicates that the model possesses a substantial German linguistic competence and is a good candidate to have acquired constraints on complex German phenomena.

Its strong general NLU abilities are further confirmed by the fact that in the overall EuroEval competition, it shares the first place with GPT-6-Astra, outperforming all Gemini's, QWEN's, DeepSeek's etc.

On the task of linker production, the LLM outperforms our constraint-based models significantly. We conclude it has a general competence in German compounding.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

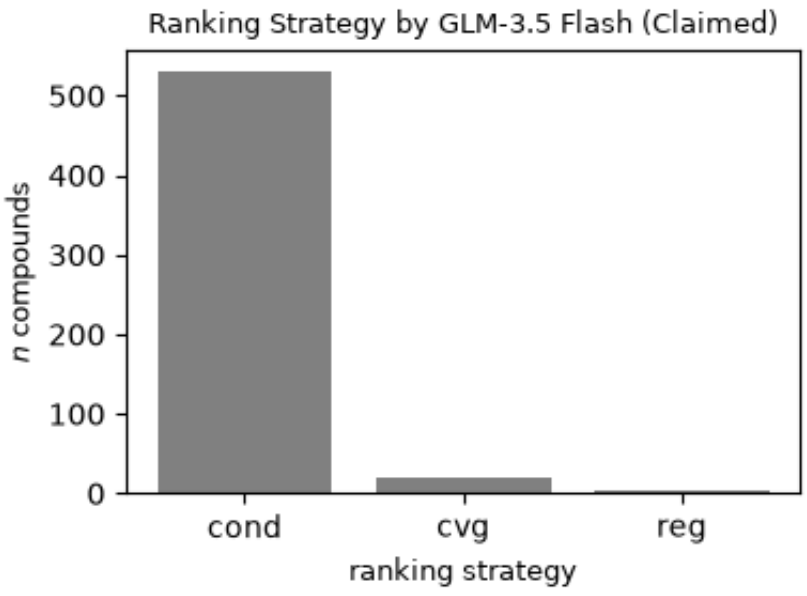
	acc	f1	N	prec	recall
avg	0.840	0.701	555.0	0.736	0.675
_	0.859	0.875	269.0	0.892	0.859
_+=er_	0.858	0.832	141.0	0.807	0.858
_+en_	0.877	0.915	122.0	0.955	0.877
_+s_	0.000	0.000	11.0	0.000	0.000
_+e_	0.333	0.400	6.0	0.500	0.333
_+=e_	0.800	0.889	5.0	1.000	0.800
_+=_	1.000	1.000	1.0	1.000	1.000

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

	acc	f1
GLM-5.3 Flash	0.701	0.840
yang-cond	0.712	0.526
yang-cvg	0.688	0.483
albright-hayes	0.638	0.242
rand	0.368	0.156
zero	0.485	0.093

The LLM itself classifies the reasoning for its rankings as predominantly condition-specificity-based. THe number of coverage- and regularity-based rankings is negligible.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.



Indeed, the rankings from the LLM are much closer to what our specificity-based models produce. Thus, $80\%$$80\%$ of the rankings from the LLM are identical to the corresponding rankings from `Y16-cvg`, $72\%$$72\%$ to those from `Y16-cond`, while the percentage of LLM rankings identical to `AH02` is only $26\%$$26\%$. Average Kendall's tau demonstrates that not only many ranking from the LLM are identical to the ranking from the specificity-based models, they also correlate highly: $0.79$$0.79$ and $0.76$$0.76$ for `Y16-cvg` and `Y16-cond`, respectively.

Moreover, the LLM consistently reasons about constraint ranking in terms of the Elsewhere Condition. Here's an example of its reasoning for 'Eigelb':

> Specificity ranking (the Elsewhere Condition): rules are ordered from most specific to most general. Rule 3 is the most specific (plural in -er AND singular meaning), Rule 2 is intermediate (plural in -er), and Rule 1 is the general default for all German compounds. A more specific rule, when it applies, overrides the general default, which serves only as a last resort.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

	acc	t_avg	t_pooled
GLM-3.5 Flash vs. AH02	0.157	0.262	0.295
GLM-3.5 Flash vs. Y16-cvg	0.800	0.788	0.819
GLM-3.5 Flash vs. Y16-cond	0.723	0.763	0.786

The fact that the LLM rankings are, however, closer to `Y16-cvg` than to `Y16-cond` is explained by the fact that the LLM consistently ranks derivational and phonological constraints higher than we do. This is one of the cases of subjective and unclear inter-category comparison, so either of us may be wrong.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

```
nation_+en_wertung
Specificity (more specific rules outrank more general rules): rule 3 targets a narrow, well-defined mor
                                                          condition  spec_score  cvg_score    reg
c_id
p2l:drv:sfx:sfx-s|def-s                                    [drv:SFX]        12.7      0.786  0.987
p2l:phon_fin:weak_F_cons$-0/s/en|def-0/s/en  [G:F] & [PL:en] & [coda:CN]        39.7      0.751  1.000
def-0                                                            NaN         0.0      0.000  0.573


haus_+=er_wand
Specificity overrides generality (the 'elsewhere' / Pāṇinian principle): the most specific applicable r
                                                 condition  spec_score  cvg_score    reg
c_id
p2l:phon_fin:sibilant$-0                [coda:S] | [coda:CLT]        12.8      0.915  0.970
p2l:decl_cl:pl:#er-0/er|!pl_interpr-0    [PL:er] & [sem:!PL]        25.2      0.829  0.622
p2l:decl_cl:pl:#er-0/er|def-0/er                    [PL:er]        14.0      0.979  0.815
def-0                                                  NaN         0.0      0.000  0.573


geschenk_idee
Specificity (precision) outranks generality: the most specific rule applying to a given noun wins, with
                          condition  spec_score  cvg_score    reg
c_id
p2l:drv:prx_deverb-s       [drv:PRX]        12.7      0.841  0.649
p2l:decl_cl:pl:#e-0|def-0     [PL:e]        14.0      0.838  0.754
def-0                            NaN         0.0      0.000  0.573


geräusch_+e_macher
Ranking by specificity and (near-)obligatoriness of the constraint. Rule 1 is the maximally general def
                                    condition  spec_score  cvg_score    reg
c_id
p2l:phon_fin:sibilant$-0   [coda:S] | [coda:CLT]        12.8      0.915  0.970
p2l:drv:prx_deverb-s                   [drv:PRX]        12.7      0.841  0.649
p2l:decl_cl:pl:#e-0|def-0                 [PL:e]        14.0      0.838  0.754
def-0                                        NaN         0.0      0.000  0.573


haus_+=er_gruppe
Specificity overrides generality: phonologically conditioned rules (avoiding a sibilant cluster like [s
                                                  condition  spec_score  cvg_score    reg
c_id
p2l:phon_fin:sibilant$-0                [coda:S] | [coda:CLT]        12.8      0.915  0.970
p2l:decl_cl:pl:#er-0/er|pl_interpr-er     [PL:er] & [sem:PL]        26.8      0.868  0.754
p2l:decl_cl:pl:#er-0/er|def-0/er                     [PL:er]        14.0      0.979  0.815
def-0                                                   NaN         0.0      0.000  0.573


geschenk_+e_liste
The ranking follows the specificity principle (a Panini-style hierarchy): the most specific rule wins o
                                     condition  spec_score  cvg_score    reg
c_id
p2l:decl_cl:pl:#e-0|pl_interpr-e  [PL:e] & [sem:PL]        26.8      0.921  0.544
p2l:drv:prx_deverb-s                     [drv:PRX]        12.7      0.841  0.649
p2l:decl_cl:pl:#e-0|def-0                   [PL:e]        14.0      0.838  0.754
def-0                                          NaN         0.0      0.000  0.573
```

The most important finding about the LLM rankings is that the linkers inferred from the ratings are much different from the linkers that the LLMs predicted during the initial production task. Thus, only $62\%$$62\%$ of generations at the ranking and at the production task are identical. The performance of the LLM from the ranking task drops to the level of the specificity-based models.

Beginnen Sie mit dem Programmieren oder generieren Sie Code mit KI.

f1_acc_df

	acc	f1	N	prec	recall
avg	0.622	0.488	555.0	0.514	0.584
_	0.758	0.686	269.0	0.626	0.758
_+=er_	0.404	0.516	141.0	0.712	0.404
_+en_	0.623	0.764	122.0	0.987	0.623
_+s_	0.000	0.000	11.0	0.000	0.000
_+e_	0.500	0.400	6.0	0.333	0.500
_+=e_	0.800	0.800	5.0	0.800	0.800
_+=_	1.000	0.250	1.0	0.143	1.000

	acc	f1
GLM-5.3 Flash (production)	0.840	0.701
yang-cond	0.712	0.526
GLM-5.3 Flash (ranking)	0.622	0.488
yang-cvg	0.688	0.483
albright-hayes	0.638	0.242
rand	0.368	0.156
zero	0.485	0.093

n oder generieren Sie Code mit KI.

Summary

The LLM showed a large gap in its performance in the production and in the ranking tasks. The former demonstrated that the model has a substantial knowledge of German compounds and is in general capable of linker prediction. The rankings revealed that the LLM is very much aware of the Elsewhere Condition, and that it consistently ranks more specific constraints higher, which is in line with the higher performance of our constraint-based models based on specificity. However, the linkers elicited from these rankings are much less accurate than those returned in the production task. While leaving much space for speculation, this finding may hint towards the fact that the model only has a factual knowledge of the compounding constraints but not any internal representations of those. To make more grounded conclusions, we need more LLMs and more interpretable experiments, as well as a more sophisticated modeling procedure.